

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Tomada de Decisão Arquitetural em Sistemas Intensivos em Dados: uma abordagem baseada em evidências

Proposta

José Bruno Barros dos Santos



São Cristóvão – Sergipe

2026

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

José Bruno Barros dos Santos

**Tomada de Decisão Arquitetural em Sistemas Intensivos em
Dados: uma abordagem baseada em evidências**

Proposta apresentada ao Programa de Pós-Graduação
em Ciência da Computação da Universidade Federal
de Sergipe como requisito parcial para a obtenção do
título de mestre em Ciência da Computação.

Orientador(a): Fábio Gomes Rocha

São Cristóvão – Sergipe

2026

We've always defined ourselves by the ability to overcome the impossible. And we count these moments. These moments when we dare to aim higher, to break barriers, to reach for the stars, to make the unknown known. We count these moments as our proudest achievements. But we lost all that. Or perhaps we've just forgotten that we are still pioneers. And we've barely begun. And that our greatest accomplishments cannot be behind us, because our destiny lies above us.

(Interstellar)

Resumo

Contexto: Sistemas intensivos em dados são fundamentais para o processamento, armazenamento, movimentação e análise de grandes volumes de dados em infraestruturas modernas de software. Esses sistemas envolvem decisões arquiteturais relacionadas a paradigmas de processamento, ingestão de dados, escalabilidade, uso de recursos e tolerância a falhas, as quais impactam diretamente atributos como *throughput*, latência, estabilidade e custo computacional.

Problema: Apesar da existência de diferentes técnicas, *frameworks* e estratégias de otimização para sistemas intensivos em dados, as evidências empíricas disponíveis na literatura permanecem fragmentadas, heterogêneas e, muitas vezes, difíceis de comparar. Essa limitação dificulta a tomada de decisão arquitetural baseada em evidências, especialmente em cenários que exigem a escolha entre processamento em lote, processamento em fluxo e diferentes configurações de execução.

Solução: Esta pesquisa propõe investigar uma abordagem quantitativa de apoio à decisão arquitetural em sistemas intensivos em dados, integrando evidências da literatura, resultados experimentais e características observáveis dos dados, como volume, entropia, redundância, cardinalidade e variabilidade.

Método: A pesquisa foi organizada em três etapas complementares: uma Revisão Sistemática da Literatura, uma campanha experimental controlada e a proposta de um modelo quantitativo de apoio à decisão arquitetural. A revisão sistemática identificou técnicas, métricas, lacunas e evidências relacionadas ao design de sistemas intensivos em dados. A campanha experimental comparou processamento *batch* e *streaming* em um ambiente baseado em Apache Spark, Spark Structured Streaming, Apache Kafka e Docker, utilizando o dataset NYC Taxi. Foram avaliados diferentes volumes de dados, taxas de eventos, números de *workers* e intervalos de *trigger*, considerando métricas como *throughput*, latência, uso de CPU, consumo de memória, *backpressure* e escalabilidade. **Sumarização dos resultados:** A revisão sistemática identificou 979 registros, dos quais dez estudos primários atenderam aos critérios de inclusão e qualidade. A síntese quantitativa exploratória indicou efeitos positivos de otimizações arquiteturais, com razão média de resposta de aproximadamente $2,7\times$ em relação às configurações de referência. Os resultados experimentais parciais indicaram que o processamento *batch* apresentou maior *throughput* nos cenários avaliados, enquanto o processamento *streaming* mostrou maior sensibilidade à taxa de eventos, ao intervalo de *trigger* e aos custos de coordenação dos micro-lotes. **Contribuição:** Espera-se que esta pesquisa contribua para a tomada de decisão arquitetural em sistemas intensivos em dados por meio de uma abordagem baseada em evidências empíricas, resultados experimentais e características informacionais dos dados, auxiliando arquitetos de software e engenheiros de dados na comparação preliminar entre alternativas arquiteturais antes da implementação completa de uma solução.

Palavras-chave: Sistemas Intensivos em Dados. Arquitetura de Software. Tomada de Decisão Arquitetural. Processamento em Fluxo. Teoria da Informação.

Abstract

Context: Data-intensive systems are fundamental for processing, storing, moving, and analyzing large volumes of data in modern software infrastructures. These systems involve architectural decisions related to processing paradigms, data ingestion, scalability, resource usage, and fault tolerance, which directly impact attributes such as *throughput*, latency, stability, and computational cost. **Problem:** Despite the existence of different techniques, *frameworks*, and optimization strategies for data-intensive systems, the empirical evidence available in the literature remains fragmented, heterogeneous, and often difficult to compare. This limitation makes evidence-based architectural decision-making difficult, especially in scenarios that require choosing between batch processing, stream processing, and different execution configurations. **Solution:** This research proposes to investigate a quantitative approach to support architectural decision-making in data-intensive systems by integrating evidence from the literature, experimental results, and observable data characteristics, such as volume, entropy, redundancy, cardinality, and variability. **Method:** The research was organized into three complementary stages: a Systematic Literature Review, a controlled experimental campaign, and the proposal of a quantitative model to support architectural decision-making. The systematic review identified techniques, metrics, gaps, and evidence related to the design of data-intensive systems. The experimental campaign compared *batch* and *streaming* processing in an environment based on Apache Spark, Spark Structured Streaming, Apache Kafka, and Docker, using the NYC Taxi dataset. Different data volumes, event rates, numbers of *workers*, and *trigger* intervals were evaluated, considering metrics such as *throughput*, latency, CPU usage, memory consumption, *backpressure*, and scalability. **Summary of results:** The systematic review identified 979 records, of which ten primary studies met the inclusion and quality criteria. The exploratory quantitative synthesis indicated positive effects of architectural optimizations, with an average response ratio of approximately $2.7\times$ compared to baseline configurations. The preliminary experimental results indicated that *batch* processing achieved higher *throughput* in the evaluated scenarios, while *streaming* processing showed greater sensitivity to event rate, *trigger* interval, and micro-batch coordination costs. **Contribution:** This research is expected to contribute to architectural decision-making in data-intensive systems through an approach based on empirical evidence, experimental results, and informational characteristics of data, supporting software architects and data engineers in the preliminary comparison of architectural alternatives before the complete implementation of a solution.

Keywords: Data-Intensive Systems. Software Architecture. Architectural Decision-Making. Stream Processing. Information Theory.

Lista de Figuras

Figura 1 – Encadeamento conceitual da pesquisa.	14
Figura 2 – Desenho metodológico da pesquisa	25
Figura 3 – Fluxo PRISMA 2020 do processo de seleção dos estudos	31
Figura 4 – Distribuição temática dos estudos incluídos	35
Figura 5 – Arquitetura geral da campanha experimental organizada em camadas funcionais.	39
Figura 6 – Variações de throughput sob escalonamento de carga de trabalho (RQ1). . .	48
Figura 7 – Evolução da variabilidade operacional ($CV\%$) conforme a escala de carga (RQ2).	49
Figura 8 – Dilema arquitetural (trade-off) entre latência do evento e backpressure operacional (RQ3).	51
Figura 9 – Curvas de speedup experimental frente ao limite linear ideal (RQ4).	53
Figura 10 – Cronograma das atividades de pesquisa.	57

Lista de Tabelas

Tabela 1 – Principais métricas de desempenho consideradas na pesquisa	21
Tabela 2 – Critérios de inclusão e exclusão da RSL	30
Tabela 3 – Distribuição dos registros identificados por base de dados	31
Tabela 4 – Dimensões avaliadas no checklist de qualidade	32
Tabela 5 – Síntese dos estudos primários incluídos na RSL	35
Tabela 6 – Variáveis independentes da campanha experimental	40
Tabela 7 – Métricas avaliadas na campanha experimental	41
Tabela 8 – Organização dos cenários experimentais	42
Tabela 9 – Comparação de throughput e testes de significância não-paramétricos (RQ1).	48
Tabela 10 – Análise de variabilidade operacional sob escalonamento volumétrico (RQ2).	49
Tabela 11 – Métricas operacionais de streaming segmentadas por intervalo de trigger (RQ3).	51
Tabela 12 – Métricas de escalabilidade e eficiência de paralelização no cluster (RQ4). . .	52
Tabela 13 – Síntese dos resultados parciais e implicações arquiteturais.	53
Tabela 14 – Atividades previstas para a continuidade da pesquisa.	59
Tabela 15 – Produção científica prevista.	60

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
ACM	Association for Computing Machinery
API	Application Programming Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
GB	Gigabyte
HPC	High Performance Computing
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
LRR	Log Response Ratio
MB	Megabyte
NDP	Near-Data Processing
NVM	Non-Volatile Memory
NYC	New York City
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PROCC	Programa de Pós-Graduação em Ciência da Computação
QA	Quality Assessment
RSL	Revisão Sistemática da Literatura
SGBD	Sistema de Gerenciamento de Banco de Dados
SQL	Structured Query Language
UFS	Universidade Federal de Sergipe
WEBIST	International Conference on Web Information Systems and Technologies
YARN	Yet Another Resource Negotiator

Lista de símbolos

X	Variável aleatória discreta associada a um atributo, evento ou conjunto de valores analisado.
x_i	Valor possível assumido pela variável aleatória X .
$p(x_i)$	Probabilidade de ocorrência do valor x_i .
$H(X)$	Entropia da variável aleatória X .
$H_{max}(X)$	Entropia máxima possível da variável aleatória X .
$R(X)$	Redundância associada à variável aleatória X .
C_D	Vetor de características de um cenário de dados ou fluxo de eventos.
V	Volume de dados considerado em um cenário experimental.
λ	Taxa de chegada de eventos em um fluxo de dados.
K	Cardinalidade de um atributo ou conjunto de dados.
σ	Variabilidade temporal dos dados ou eventos.
L_{req}	Restrição de latência requerida para um cenário.
A	Conjunto de alternativas arquiteturais disponíveis.
a	Alternativa arquitetural pertencente ao conjunto A .
P_a	Vetor de desempenho associado à alternativa arquitetural a .
T_a	<i>Throughput</i> observado ou estimado para a alternativa a .
L_a	Latência observada ou estimada para a alternativa a .
$U_{cpu,a}$	Uso de CPU associado à alternativa arquitetural a .
$U_{mem,a}$	Consumo de memória associado à alternativa arquitetural a .
B_a	Nível de <i>backpressure</i> associado à alternativa arquitetural a .
E_a	Indicador de eficiência ou estabilidade associado à alternativa arquitetural a .
$S(a)$	Escore de adequação atribuído à alternativa arquitetural a .

w_i	Peso atribuído ao critério i em uma função de decisão.
RR_i	Razão de resposta do estudo ou cenário i .
LRR_i	Razão de resposta logarítmica do estudo ou cenário i .
$\overline{LRR}$	Média dos valores de Razão de Resposta Logarítmica.
k	Número de estudos incluídos na síntese quantitativa.
$M_{tratamento}$	Valor da métrica observado na configuração otimizada ou experimental.
$M_{baseline}$	Valor da métrica observado na configuração de referência.
$N_{registros}$	Número de registros processados em uma execução experimental.
$T_{processamento}$	Tempo total de processamento de uma execução experimental.
δ	Tamanho de efeito de Cliff's Delta.

Sumário

1	Introdução	12
1.1	Objetivos	14
2	Fundamentação Teórica	16
2.1	Sistemas Intensivos em Dados e Decisões Arquiteturais	16
2.2	Processamento em Lote e Processamento em Fluxo	17
2.3	Apache Spark, Spark Structured Streaming e Apache Kafka	18
2.4	Métricas de Desempenho	20
2.5	Teoria da Informação e Apoio à Decisão Arquitetural	21
3	Metodologia	23
3.1	Caracterização da Pesquisa	23
3.2	Desenho Metodológico	24
3.3	Procedimentos de Análise	25
3.4	Validade da Pesquisa	26
4	Revisão Sistemática da Literatura	28
4.1	Objetivo e Questões de Pesquisa	28
4.2	Estratégia de Busca	29
4.3	CrITÉrios de Inclusão e Exclusão	30
4.4	Processo de Seleção dos Estudos	30
4.5	Avaliação da Qualidade	32
4.6	Extração dos Dados	32
4.7	Síntese Quantitativa Exploratória	33
4.8	Estudos Incluídos e Resultados	34
4.9	Distribuição Temática dos Estudos	34
4.10	Lacunas Identificadas	36
5	Campanha Experimental	37
5.1	Objetivo e Questões Experimentais	37
5.2	Ambiente Experimental	38
5.3	Dataset e Preparação dos Dados	39
5.4	Variáveis e Métricas	40
5.5	Cenários Experimentais	41
5.6	Execução e Repetições	42
5.7	Automação e Coleta de Métricas	43

5.8	Procedimento de Análise Estatística	43
6	Resultados Parciais	45
6.1	Resultados da Revisão Sistemática da Literatura	45
6.2	Resultados da Síntese Quantitativa Exploratória	46
6.3	Resultados da Campanha Experimental	47
6.3.1	RQ1 — Paradigmas de Processamento e Vazão Máxima	47
6.3.2	RQ2 — Impacto do Volume de Dados e Variabilidade Operacional . . .	48
6.3.3	RQ3 — Intervalos de Trigger e Latência de Eventos	50
6.3.4	RQ4 — Escalabilidade Horizontal e Eficiência Arquitetural	52
6.4	Integração dos Resultados	54
6.5	Implicações para o Modelo de Apoio à Decisão	54
7	Cronograma	56
8	Próximos Passos	58
8.1	Atividades Necessárias para a Conclusão da Pesquisa	58
8.2	Refinamento do Modelo de Apoio à Decisão Arquitetural	58
8.3	Cálculo das Características Informacionais dos Dados	59
8.4	Produção Científica Prevista	59
8.5	Delimitação do Escopo e Trabalhos Futuros	59
	Referências Bibliográficas	61

1

Introdução

A crescente digitalização de processos sociais, científicos, industriais e organizacionais tem provocado um aumento contínuo na produção, armazenamento, transmissão e processamento de dados. Esse cenário intensifica a relevância dos sistemas intensivos em dados, isto é, sistemas projetados para lidar com grandes volumes de dados, múltiplas fontes de informação, diferentes formatos e requisitos rigorosos de desempenho, escalabilidade, disponibilidade e confiabilidade. Em ambientes de Big Data, esses desafios são frequentemente associados às características de volume, velocidade, variedade e veracidade dos dados, as quais influenciam diretamente o modo como as arquiteturas de software são concebidas e avaliadas (DAI et al., 2019; HILBERT, 2016).

Diferentemente de sistemas tradicionais, aplicações intensivas em dados dependem da integração entre componentes distribuídos, mecanismos de armazenamento, plataformas de processamento paralelo, sistemas de mensageria, estratégias de movimentação de dados e técnicas de gerenciamento de recursos. Nesse contexto, decisões arquiteturais deixam de ser apenas escolhas estruturais de alto nível e passam a exercer impacto direto sobre atributos de qualidade, como latência, *throughput*, escalabilidade, tolerância a falhas e eficiência no uso de recursos computacionais (MATTMANN et al., 2004; JI et al., 2020; KLEPPMANN, 2017).

Entre as decisões arquiteturais mais relevantes nesse domínio, destacam-se aquelas relacionadas à forma como os dados são ingeridos, movimentados, armazenados e processados ao longo de um *pipeline*. A escolha entre processamento em lote (*batch processing*) e processamento em fluxo (*stream processing*), por exemplo, pode alterar substancialmente o comportamento do sistema em termos de vazão, tempo de resposta, estabilidade temporal e consumo de recursos. Do mesmo modo, decisões sobre particionamento, escalonamento, localização do processamento, uso de memória, transferência entre sistemas e configuração de mecanismos de tolerância a falhas podem produzir impactos distintos conforme o volume dos dados, a taxa de chegada dos eventos e o ambiente de execução (HAYNES; CHEUNG; BALAZINSKA, 2016; WANG et al.,

2018; AUNG; MIN; MAW, 2019; LYU et al., 2022).

Apesar da existência de diversas técnicas, *frameworks* e estratégias voltadas à otimização de sistemas intensivos em dados, a literatura ainda apresenta evidências empíricas fragmentadas. Muitos estudos avaliam soluções específicas em contextos particulares, utilizando métricas, *workloads* e ambientes experimentais heterogêneos. Essa fragmentação dificulta a comparação entre abordagens, reduz a capacidade de generalização dos resultados e limita o apoio sistemático à tomada de decisão arquitetural. Como consequência, arquitetos de software frequentemente precisam decidir entre alternativas tecnológicas e arquiteturais com base em experiências prévias, recomendações práticas ou experimentos isolados, sem dispor de mecanismos quantitativos capazes de orientar essas escolhas de forma mais objetiva.

Diante desse contexto, esta pesquisa parte do seguinte problema: como apoiar, de forma sistemática e baseada em evidências, a tomada de decisão arquitetural em sistemas intensivos em dados, considerando características dos dados, estratégias de processamento e métricas observáveis de desempenho?

Para enfrentar essa lacuna, a pesquisa foi organizada em três movimentos complementares. O primeiro consistiu na realização de uma Revisão Sistemática da Literatura sobre o *design* de sistemas intensivos em dados, acompanhada de uma síntese quantitativa exploratória. Essa etapa permitiu identificar técnicas arquiteturais recorrentes, métricas utilizadas na avaliação de desempenho, padrões de otimização e lacunas metodológicas relacionadas ao relato experimental.

O segundo movimento consistiu na condução de uma campanha experimental voltada à produção de evidência empírica controlada sobre decisões arquiteturais em sistemas intensivos em dados. O experimento comparou os paradigmas de processamento *batch* e *streaming* utilizando tecnologias amplamente adotadas, como Apache Spark, Spark Structured Streaming e Apache Kafka. Foram avaliados diferentes volumes de dados, taxas de eventos, números de *workers* e configurações de *trigger*, considerando métricas como *throughput*, latência, uso de CPU, consumo de memória, *backpressure* e taxa de processamento (ZAHARIA, 2016; ARMBRUST et al., 2018; KREPS; NARKHEDE; RAO, 2011; MATTEUSSI et al., 2022).

Os resultados experimentais obtidos reforçam a importância de compreender os *trade-offs* envolvidos nas decisões arquiteturais. O processamento *batch* tende a favorecer maiores taxas de *throughput* em determinados cenários, enquanto o processamento em fluxo introduz custos associados à ingestão contínua, à coordenação dos micro-lotes e ao controle de estabilidade temporal. Além disso, fatores como volume de dados, paralelismo horizontal e intervalo de *trigger* podem alterar significativamente o comportamento do sistema, indicando que decisões arquiteturais devem ser analisadas de forma contextualizada e baseada em evidências.

O terceiro movimento da pesquisa consiste na proposta de uma abordagem quantitativa de apoio à decisão arquitetural, considerando evidências empíricas e características informacionais

dos dados. A intenção é investigar se propriedades como volume, taxa de eventos, entropia, redundância e variabilidade podem ser combinadas com métricas de desempenho para apoiar a comparação entre alternativas arquiteturais.

A Figura 1 apresenta o encadeamento geral da pesquisa.

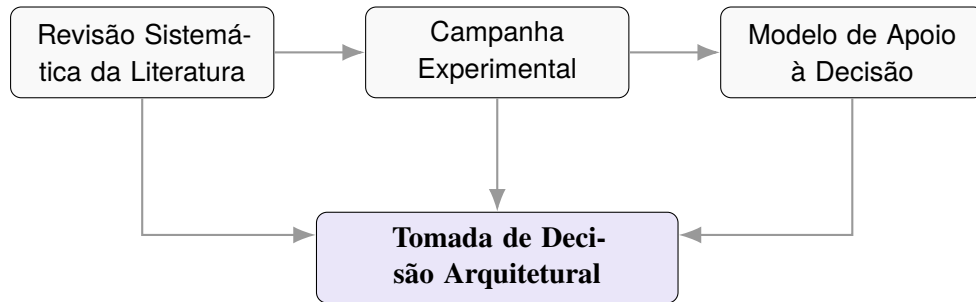


Figura 1 – Encadeamento conceitual da pesquisa.

1.1 Objetivos

O objetivo geral desta pesquisa é investigar como evidências provenientes da literatura científica, resultados experimentais e características observáveis dos dados podem ser integrados para apoiar a tomada de decisão arquitetural em sistemas intensivos em dados.

Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

1. caracterizar, por meio de uma Revisão Sistemática da Literatura, as principais técnicas, abordagens, métricas e lacunas relacionadas ao design de sistemas intensivos em dados;
2. sintetizar evidências quantitativas sobre o impacto de otimizações arquiteturais e sistêmicas no desempenho de sistemas intensivos em dados;
3. projetar e executar uma campanha experimental para comparar estratégias de processamento *batch* e *streaming* sob diferentes condições operacionais;
4. analisar estatisticamente os resultados experimentais, considerando métricas como *throughput*, latência, escalabilidade, uso de recursos e estabilidade do processamento;
5. propor uma abordagem quantitativa de apoio à decisão arquitetural que relacione características dos dados e do processamento ao comportamento esperado do sistema.

A justificativa desta pesquisa reside na necessidade de reduzir a dependência exclusiva de avaliações empíricas isoladas no processo de decisão arquitetural. Em sistemas intensivos em dados, a realização de experimentos completos para cada alternativa arquitetural pode demandar tempo, infraestrutura computacional e esforço de implementação significativos. Dessa forma, uma abordagem baseada em evidências pode auxiliar arquitetos de software na análise preliminar

de alternativas, contribuindo para decisões mais fundamentadas, rastreáveis e alinhadas aos requisitos de desempenho e escalabilidade do sistema.

Além disso, esta pesquisa contribui para a área de Engenharia de Software ao aproximar três dimensões complementares: a consolidação de evidências científicas por meio de revisão sistemática, a produção de evidências empíricas por meio de experimentação controlada e a construção de mecanismos quantitativos para apoiar decisões arquiteturais. Essa integração busca avançar o conhecimento sobre o design de sistemas intensivos em dados e fornecer subsídios para pesquisas futuras relacionadas à avaliação, predição e recomendação de arquiteturas orientadas ao processamento intensivo de dados.

Esta proposta está organizada em oito capítulos. O Capítulo 2 apresenta a fundamentação teórica relacionada a sistemas intensivos em dados, decisões arquiteturais, processamento *batch* e *streaming*, Apache Spark, Apache Kafka e Teoria da Informação. O Capítulo 3 descreve a metodologia da pesquisa. O Capítulo 4 apresenta a Revisão Sistemática da Literatura conduzida nesta pesquisa. O Capítulo 5 descreve a campanha experimental realizada. O Capítulo 6 apresenta os resultados parciais. O Capítulo 7 apresenta o cronograma da pesquisa. Por fim, o Capítulo 8 descreve os próximos passos e as atividades previstas para continuidade da dissertação.

2

Fundamentação Teórica

Os fundamentos conceituais que sustentam a pesquisa envolvem sistemas intensivos em dados, decisões arquiteturais e estratégias de processamento em larga escala. Inicialmente, discutem-se os sistemas intensivos em dados e sua relação com decisões arquiteturais. Em seguida, examinam-se os paradigmas de processamento em lote e processamento em fluxo, com ênfase em seus principais *trade-offs*. Também são abordadas as tecnologias utilizadas na campanha experimental, especialmente Apache Spark, Spark Structured Streaming e Apache Kafka. Por fim, discutem-se as métricas de desempenho consideradas na avaliação experimental e os conceitos da Teoria da Informação que fundamentam a proposta de apoio à decisão arquitetural.

2.1 Sistemas Intensivos em Dados e Decisões Arquiteturais

Sistemas intensivos em dados são sistemas de software cujo funcionamento depende fortemente da coleta, armazenamento, processamento, movimentação e análise de grandes volumes de dados. Esses sistemas são caracterizados não apenas pela quantidade de dados manipulada, mas também pela diversidade de formatos, pela velocidade com que os dados são produzidos e pela necessidade de manter níveis aceitáveis de desempenho, escalabilidade, confiabilidade e disponibilidade.

O crescimento de aplicações em domínios como computação em nuvem, Internet das Coisas (IoT), ciência de dados, sistemas de recomendação, monitoramento em tempo real e análise de grandes volumes de registros operacionais ampliou a importância desse tipo de sistema. Em tais contextos, os dados passam a ocupar papel central na arquitetura, influenciando decisões sobre armazenamento, particionamento, replicação, processamento, integração entre componentes e comunicação entre serviços (DAI et al., 2019; HILBERT, 2016; KLEPPMANN, 2017).

Diferentemente de sistemas transacionais tradicionais, nos quais a lógica de negócio

pode ocupar papel predominante na estrutura arquitetural, sistemas intensivos em dados exigem atenção especial ao fluxo dos dados ao longo da arquitetura. A forma como os dados são ingeridos, transformados, transportados e persistidos pode afetar diretamente atributos de qualidade. Por esse motivo, decisões arquiteturais nesse tipo de sistema devem considerar tanto os requisitos funcionais quanto os requisitos de qualidade associados ao comportamento operacional da aplicação (MATTMANN et al., 2004; JI et al., 2020).

Decisões arquiteturais correspondem a escolhas de projeto que afetam a estrutura, o comportamento e os atributos de qualidade de um sistema de software. Em sistemas intensivos em dados, essas decisões envolvem, entre outros aspectos, a escolha de estratégias de processamento, mecanismos de armazenamento, formas de particionamento, políticas de replicação, tecnologias de mensageria, modelos de consistência, escalonamento de tarefas e alocação de recursos computacionais.

Essas decisões são críticas porque impactam diretamente características como latência, *throughput*, escalabilidade, custo computacional, tolerância a falhas e eficiência no uso de CPU, memória, disco e rede. Por exemplo, uma arquitetura pode priorizar maior vazão em processamento em lote, enquanto outra pode priorizar menor latência e maior responsividade em processamento contínuo. Da mesma forma, uma estratégia de movimentação de dados pode reduzir o tempo de transferência entre sistemas, enquanto uma estratégia de processamento próximo aos dados pode reduzir custos de I/O e melhorar a escalabilidade (HAYNES; CHEUNG; BALAZINSKA, 2016; VINÇON et al., 2020).

A literatura apresenta diferentes propostas voltadas à otimização de aspectos específicos dessas arquiteturas. Há estudos relacionados à otimização de escalonamento em ambientes Spark sobre YARN (WANG et al., 2018), à compilação dinâmica de consultas SQL em Apache Spark (SCHIAVIO; BONETTA; BINDER, 2020), ao gerenciamento fino de recursos em ambientes de Big Data (LYU et al., 2022), ao uso de armazenamento sensível a NVM para cargas analíticas (GÖTZE; BAUMANN; SATTLER, 2018) e à ingestão transacional escalável de dados (LI et al., 2022). Embora esses estudos apresentem contribuições relevantes, muitos deles avaliam soluções em contextos específicos, com métricas e configurações experimentais distintas.

Essa heterogeneidade torna a tomada de decisão arquitetural uma tarefa complexa. Arquitetos de software precisam escolher entre alternativas que podem apresentar vantagens em determinados cenários e limitações em outros. Assim, uma decisão adequada depende não apenas do conhecimento sobre tecnologias disponíveis, mas também da compreensão dos dados, da carga de trabalho, dos requisitos de desempenho e das restrições operacionais do ambiente.

2.2 Processamento em Lote e Processamento em Fluxo

Entre as decisões arquiteturais mais relevantes em sistemas intensivos em dados está a escolha entre processamento em lote e processamento em fluxo. O processamento em lote, ou

batch processing, consiste na execução de tarefas sobre conjuntos de dados previamente armazenados. Esse paradigma é amplamente utilizado em análises periódicas, geração de relatórios, processamento histórico e execução de cargas analíticas sobre grandes volumes de dados.

O processamento em fluxo, ou *stream processing*, por sua vez, é orientado ao processamento contínuo de eventos à medida que eles são produzidos. Esse paradigma é especialmente relevante em cenários que exigem respostas próximas ao tempo real, como monitoramento de sensores, análise de *logs*, detecção de anomalias, rastreamento de atividades e integração contínua de eventos em *pipelines* de dados (NASIRI; NASEHI; GOUDARZI, 2019; ARMBRUST et al., 2018).

A escolha entre esses paradigmas envolve *trade-offs*. O processamento em lote pode se beneficiar da disponibilidade de todo o conjunto de dados antes da execução, permitindo otimizações globais, melhor amortização de custos fixos e maior aproveitamento de recursos em determinados cenários. Por outro lado, esse paradigma pode não ser adequado para aplicações que exigem baixa latência ou resposta contínua a eventos.

O processamento em fluxo reduz o intervalo entre a chegada dos dados e sua análise, mas introduz desafios adicionais. Entre eles estão a necessidade de lidar com ingestão contínua, controle de atrasos, variação na taxa de eventos, tolerância a falhas, gerenciamento de estado, pressão de retorno (*backpressure*) e estabilidade temporal do *pipeline*. Em *frameworks* baseados em micro-lotes, como o Spark Structured Streaming, o intervalo de *trigger* também influencia o comportamento do sistema, afetando latência, *throughput* e variabilidade do processamento (ARMBRUST et al., 2018; IVANOV; TAAFFE, 2018; MATTEUSSI et al., 2022).

A literatura sobre processamento de fluxos mostra que sistemas baseados em micro-lotes e sistemas de processamento contínuo apresentam compromissos distintos. Sistemas de micro-lotes tendem a simplificar mecanismos de tolerância a falhas e integração com processamento em lote, mas podem introduzir maior latência em função do agrupamento dos dados em intervalos discretos. Sistemas contínuos podem alcançar menor latência, porém geralmente envolvem maior complexidade no controle de estado, recuperação e adaptação a falhas (VENKATARAMAN et al., 2017).

2.3 Apache Spark, Spark Structured Streaming e Apache Kafka

A campanha experimental desta pesquisa utiliza tecnologias amplamente empregadas em arquiteturas de processamento distribuído e ingestão de dados. Entre elas, destacam-se o Apache Spark, o Spark Structured Streaming e o Apache Kafka.

O Apache Spark é um *framework* de processamento distribuído projetado para executar aplicações de dados em larga escala sobre *clusters* computacionais. Sua arquitetura oferece

suporte a diferentes tipos de carga, incluindo processamento em lote, consultas SQL, aprendizado de máquina, processamento de grafos e processamento de fluxos de dados. O Spark tornou-se uma das plataformas mais utilizadas para processamento distribuído devido à sua flexibilidade, ao uso de abstrações de alto nível e à integração com diferentes formatos e fontes de dados (ZAHARIA, 2016; SHANAHAN; DAI, 2015).

No contexto do processamento em lote, o Spark permite executar operações distribuídas sobre grandes conjuntos de dados, explorando paralelismo e distribuição de tarefas entre múltiplos nós do *cluster*. Estudos mostram que o volume de dados, o número de recursos disponíveis e a configuração do ambiente podem afetar significativamente o desempenho de aplicações baseadas em Spark (AWAN et al., 2015; AHMED et al., 2020).

Além do volume de dados, a alocação de recursos computacionais é um fator relevante para o desempenho de aplicações Spark. A quantidade de memória atribuída aos executores, o número de núcleos, o número de nós, o paralelismo e a configuração do ambiente podem influenciar o tempo de execução, utilização média dos nós, gargalos de memória, operações de I/O e coleta de lixo (*garbage collection*). Estudos experimentais indicam que diferentes combinações de tamanho dos dados, número de nós e recursos dos executores podem exigir configurações distintas para alcançar melhores resultados (ALCANTARA, 2019; Shyamasundar L. B.; V. Anilkumar; Jhansi Rani P., 2019).

O Spark Structured Streaming estende o modelo do Spark para aplicações de processamento contínuo. Sua principal característica é oferecer uma API declarativa para fluxos de dados, permitindo que consultas sobre dados em tempo real sejam expressas de maneira semelhante a consultas sobre dados estáticos. Internamente, o Structured Streaming utiliza um modelo baseado em micro-lotes, no qual os dados recebidos são agrupados em pequenos intervalos de tempo e processados de forma incremental (ARMBRUST et al., 2018).

Esse modelo simplifica o desenvolvimento de aplicações de *streaming*, mas também introduz custos associados à coordenação dos micro-lotes, ao gerenciamento de estado, ao controle do intervalo de *trigger* e à sincronização entre componentes. Por isso, a configuração adequada do ambiente e dos parâmetros de execução é essencial para alcançar o equilíbrio entre *throughput*, latência e estabilidade do processamento. Estudos exploratórios sobre Spark Structured Streaming mostram que fatores como variação de volume, velocidade dos dados, número de consultas paralelas e intervalo de processamento podem alterar o comportamento do sistema (IVANOV; TAAFFE, 2018).

O Apache Kafka, por sua vez, é um sistema distribuído de mensageria baseado em *log*, projetado para ingestão, armazenamento temporário e distribuição de fluxos de eventos em larga escala. O Kafka organiza mensagens em tópicos particionados, permitindo que produtores publiquem eventos e consumidores os leiam de forma escalável e desacoplada. Essa arquitetura favorece a construção de *pipelines* de dados distribuídos, nos quais diferentes componentes podem produzir e consumir eventos de maneira independente (KREPS; NARKHEDE; RAO,

2011).

Em sistemas intensivos em dados, o Kafka é frequentemente utilizado como camada de ingestão e transporte de eventos. Sua capacidade de lidar com altas taxas de mensagens, particionamento e replicação o torna adequado para arquiteturas orientadas a eventos e *pipelines* de processamento em fluxo. No entanto, seu uso também envolve decisões relacionadas ao número de partições, replicação, taxa de produção, consumo, persistência, tolerância a falhas e integração com *frameworks* de processamento.

A confiabilidade em *pipelines* baseados em Kafka pode ser influenciada por mecanismos de *checkpoint*, replicação e recuperação após falhas. Estudos sobre arquiteturas de *pipelines* Kafka indicam que mecanismos de *checkpoint* podem reduzir o tempo de recuperação e perdas de mensagens em cenários de falha, reforçando a importância de decisões arquiteturais relacionadas à tolerância a falhas em sistemas de processamento contínuo (AUNG; MIN; MAW, 2019).

2.4 Métricas de Desempenho

A avaliação de sistemas intensivos em dados depende da definição de métricas capazes de representar o comportamento do sistema sob diferentes condições operacionais. Entre as métricas mais utilizadas estão *throughput*, latência, tempo total de execução, uso de CPU, consumo de memória, utilização de disco, tráfego de rede, taxa de processamento e indicadores de estabilidade.

O *throughput* representa a quantidade de dados processada por unidade de tempo. Em experimentos com processamento de dados, essa métrica pode ser expressa, por exemplo, em linhas por segundo, eventos por segundo ou *bytes* por segundo. A latência, por sua vez, representa o tempo necessário para que um dado evento seja processado ou para que uma operação produza resultado. Em sistemas de *streaming*, a latência pode ser analisada considerando o tempo entre a chegada do evento e seu processamento efetivo.

O uso de CPU e memória permite avaliar a eficiência do sistema em relação aos recursos computacionais disponíveis. Em ambientes distribuídos, tais métricas são importantes para compreender se o aumento do paralelismo resulta em ganhos efetivos ou se introduz *overhead* de coordenação. Além disso, em *pipelines* de *streaming*, o *backpressure* representa um indicador relevante de sobrecarga, pois indica a incapacidade temporária do sistema de acompanhar a taxa de chegada dos dados.

A literatura mostra que a avaliação de *frameworks* distribuídos frequentemente considera métricas como tempo de execução, utilização de recursos, escalabilidade horizontal, escalabilidade vertical e custo. Em ambientes de nuvem e processamento distribuído, essas métricas ajudam a compreender não apenas o desempenho bruto do sistema, mas também sua eficiência operacional diante de diferentes configurações de infraestrutura (ULLAH et al., 2024).

No contexto específico de Spark e Spark Streaming, métricas como tempo de execução, *throughput*, *speedup*, uso de memória, utilização de CPU, latência e *backpressure* são recorrentes em estudos experimentais. Pesquisas sobre desempenho em Spark indicam que o aumento do volume de dados pode intensificar custos de I/O, uso de memória e *garbage collection*, enquanto estudos sobre *streaming* mostram que variações na taxa de chegada dos eventos e na capacidade de processamento podem afetar diretamente a estabilidade do *pipeline* (AWAN et al., 2015; AHMED et al., 2020; MATTEUSSI et al., 2022).

A Tabela 1 apresenta uma síntese das principais métricas consideradas nesta pesquisa.

Tabela 1 – Principais métricas de desempenho consideradas na pesquisa

Categoria	Métrica	Foco da Avaliação
Desempenho	<i>Throughput</i>	Taxa de processamento por unidade de tempo (ex: eventos/s).
	Latência	Tempo de resposta ou atraso no processamento de um evento.
	Tempo total	Duração total do ciclo de execução (exclusivo para lote).
Recursos	Uso de CPU	Eficiência de processamento e paralelismo do <i>cluster</i> .
	Memória	Consumo de RAM e impacto de <i>garbage collection</i> .
Sustentabilidade	<i>Backpressure</i>	Saturação do <i>pipeline</i> perante a taxa de ingestão.
	Escalabilidade	Comportamento do sistema ao expandir dados ou nós.

Fonte: elaborado pelo autor.

A escolha adequada das métricas é essencial para comparar alternativas arquiteturais. Em sistemas intensivos em dados, uma abordagem pode apresentar maior *throughput*, mas também maior latência; outra pode reduzir o tempo de resposta, mas aumentar o consumo de recursos. Portanto, a avaliação deve considerar múltiplas métricas e interpretar os resultados de forma contextualizada, levando em conta os requisitos e restrições do sistema analisado.

2.5 Teoria da Informação e Apoio à Decisão Arquitetural

A Teoria da Informação, proposta por Shannon, fornece uma base matemática para medir informação, incerteza, redundância e capacidade de comunicação. Em sua formulação clássica, Shannon define a informação em termos probabilísticos, utilizando uma medida logarítmica associada ao conjunto de mensagens possíveis produzidas por uma fonte (SHANNON, 1948).

Um dos conceitos centrais dessa teoria é a entropia. Para uma variável aleatória discreta X , com possíveis valores x_i e probabilidades $p(x_i)$, a entropia pode ser expressa como:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (2.1)$$

A entropia mede o grau de incerteza associado a uma distribuição de probabilidades. Quando os eventos são igualmente prováveis, a entropia tende a ser maior, indicando maior incerteza. Quando a distribuição é concentrada em poucos eventos, a entropia tende a ser menor, indicando maior previsibilidade. A partir desse conceito, também é possível discutir noções como redundância e conteúdo informacional.

Nesta pesquisa, a Teoria da Informação é considerada como fundamento para investigar se características informacionais dos dados podem apoiar decisões arquiteturais em sistemas intensivos em dados. A hipótese é que propriedades como entropia, redundância, cardinalidade e variabilidade dos dados podem fornecer indícios sobre o comportamento esperado de diferentes estratégias de processamento, movimentação e armazenamento.

Assim, a Teoria da Informação não é utilizada para substituir a avaliação experimental, mas para complementar as evidências empíricas obtidas. Seu papel é fornecer uma base quantitativa que permita explorar relações entre características dos dados e atributos observáveis de desempenho, contribuindo para a construção de um modelo de apoio à decisão arquitetural. Dessa forma, os conceitos apresentados neste capítulo sustentam as etapas seguintes da pesquisa, especialmente a Revisão Sistemática da Literatura, a campanha experimental e a proposta de modelagem quantitativa.

3

Metodologia

O percurso metodológico adotado na pesquisa articula três dimensões complementares: uma Revisão Sistemática da Literatura (RSL), uma campanha experimental controlada e a proposta de um modelo quantitativo de apoio à decisão arquitetural. Essa organização permite combinar evidências secundárias, obtidas a partir da literatura científica, com evidências primárias, obtidas por meio de experimentação, buscando fundamentar a tomada de decisão arquitetural em sistemas intensivos em dados.

A estratégia metodológica adotada aproxima-se da perspectiva da Engenharia de Software Baseada em Evidências, na qual decisões, práticas e recomendações devem ser apoiadas por evidências sistematicamente coletadas, avaliadas e interpretadas (KITCHENHAM; BUDGEN; BRERETON, 2015). Nesse sentido, a Revisão Sistemática da Literatura contribui para organizar o conhecimento existente, enquanto a campanha experimental permite produzir evidências empíricas próprias sobre decisões arquiteturais específicas. A integração dessas etapas fornece a base para a proposta de um modelo quantitativo de apoio à decisão.

3.1 Caracterização da Pesquisa

Esta pesquisa pode ser caracterizada como aplicada, uma vez que busca produzir conhecimento voltado ao apoio da tomada de decisão arquitetural em sistemas intensivos em dados. O objetivo não é apenas compreender um fenômeno técnico, mas também propor uma forma de apoiar escolhas arquiteturais relacionadas ao processamento, movimentação e análise de grandes volumes de dados.

Quanto aos objetivos, a pesquisa possui caráter exploratório e explicativo. É exploratória porque investiga relações entre características dos dados, estratégias de processamento e métricas de desempenho ainda pouco sistematizadas na literatura. Também é explicativa porque busca compreender como determinadas decisões arquiteturais influenciam o comportamento

observado em ambientes de processamento intensivo de dados, especialmente em cenários que envolvem processamento em lote e processamento em fluxo.

Quanto à abordagem, a pesquisa combina métodos qualitativos e quantitativos. A dimensão qualitativa está presente na Revisão Sistemática da Literatura, por meio da identificação, categorização e interpretação das técnicas arquiteturais, métricas e lacunas presentes nos estudos primários. A dimensão quantitativa aparece tanto na síntese exploratória dos efeitos reportados na literatura quanto na análise estatística dos resultados experimentais.

Do ponto de vista dos procedimentos técnicos, a pesquisa envolve revisão sistemática, análise documental, experimentação controlada e modelagem quantitativa. A revisão sistemática foi utilizada para consolidar evidências existentes; a experimentação foi conduzida para avaliar empiricamente decisões arquiteturais específicas; e a modelagem quantitativa será utilizada para propor uma abordagem de apoio à decisão baseada em características observáveis dos dados e do processamento.

3.2 Desenho Metodológico

A pesquisa foi estruturada em três etapas principais. A primeira etapa corresponde à Revisão Sistemática da Literatura, cujo objetivo foi identificar técnicas, abordagens, métricas e lacunas relacionadas ao *design* de sistemas intensivos em dados. Essa etapa fornece a base conceitual e empírica para compreender o estado da arte e delimitar o problema investigado. A condução da revisão seguiu diretrizes consolidadas para revisões sistemáticas em Engenharia de Software (KITCHENHAM; CHARTERS, 2007; KITCHENHAM et al., 2009; KITCHENHAM; BUDGEN; BRERETON, 2015) e utilizou o protocolo PRISMA 2020 para relatar o processo de identificação, triagem, elegibilidade e inclusão dos estudos (PAGE et al., 2021).

A segunda etapa corresponde à campanha experimental. Essa etapa foi planejada a partir das lacunas identificadas na literatura e teve como objetivo avaliar empiricamente o comportamento de estratégias de processamento *batch* e *streaming* em diferentes condições operacionais. O desenho experimental foi orientado por princípios de experimentação em Engenharia de Software, considerando definição de objetivos, planejamento dos fatores experimentais, identificação das variáveis, execução controlada, coleta sistemática de métricas e análise estatística dos resultados (WOHLIN et al., 2012). O experimento utilizou tecnologias amplamente adotadas em *pipelines* de dados, incluindo Apache Spark, Spark Structured Streaming e Apache Kafka (ZAHARIA, 2016; ARMBRUST et al., 2018; KREPS; NARKHEDE; RAO, 2011).

A terceira etapa consiste na proposta de um modelo quantitativo de apoio à decisão arquitetural. Essa etapa busca investigar como evidências da literatura, resultados experimentais e características informacionais dos dados podem ser integrados em uma abordagem capaz de apoiar a comparação entre alternativas arquiteturais. Para isso, serão considerados conceitos da Teoria da Informação, especialmente entropia e redundância, como possíveis indicadores para

caracterizar dados e relacioná-los a métricas observáveis de desempenho (SHANNON, 1948).

A Figura 2 apresenta uma visão geral do desenho metodológico da pesquisa.

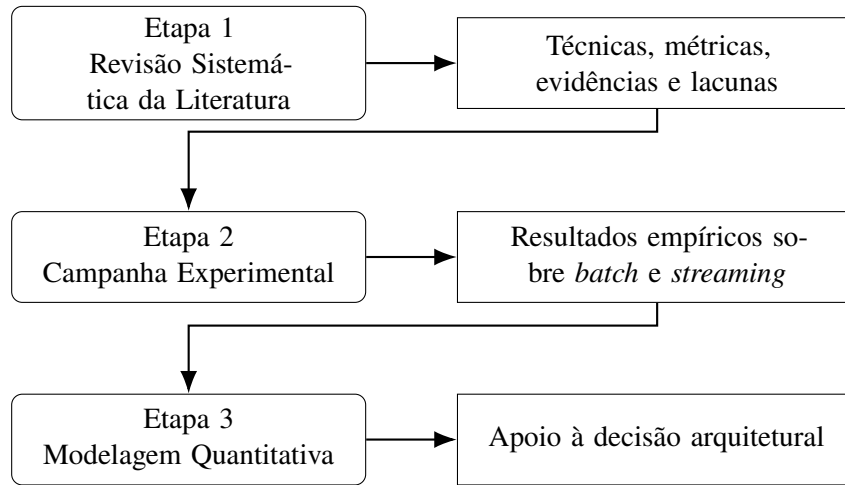


Figura 2 – Desenho metodológico da pesquisa

A Revisão Sistemática da Literatura será detalhada no Capítulo 4, enquanto a campanha experimental será apresentada no Capítulo 5. Neste capítulo, o objetivo é apresentar a lógica metodológica geral que conecta essas etapas e justifica sua integração.

3.3 Procedimentos de Análise

A análise da Revisão Sistemática da Literatura foi conduzida em duas dimensões. A primeira dimensão foi qualitativa, envolvendo a leitura, classificação e categorização dos estudos primários. Essa análise permitiu identificar técnicas de *design*, tipos de otimização, métricas utilizadas, estilos arquiteturais recorrentes e lacunas metodológicas. A segunda dimensão foi quantitativa e exploratória, utilizando o *Log Response Ratio* (LRR) como medida normalizada de efeito. Essa escolha permitiu comparar estudos que utilizaram diferentes métricas de desempenho, como tempo de execução, latência, *throughput* e *speedup*.

A análise da campanha experimental foi conduzida a partir das métricas coletadas durante as execuções. Entre as métricas consideradas estão tempo total de execução, *throughput*, latência, uso de CPU, consumo de memória, *backpressure*, taxa de processamento e métricas de entrada e saída dos contêineres. Essas métricas permitiram avaliar o comportamento dos paradigmas *batch* e *streaming* sob diferentes volumes de dados, taxas de eventos, números de *workers* e intervalos de *trigger*.

Para verificar se as diferenças observadas entre cenários eram estatisticamente significativas, foram utilizados procedimentos estatísticos não paramétricos. Inicialmente, foi aplicado o teste de Shapiro-Wilk para avaliar a normalidade das distribuições das métricas coletadas (SHAPIRO; WILK, 1965). Considerando que distribuições de desempenho em sistemas compu-

tacionais nem sempre podem ser assumidas como normalmente distribuídas, foram utilizados testes não paramétricos nas comparações entre cenários.

Para comparações entre dois grupos independentes, foi aplicado o teste de Mann-Whitney U (MANN; WHITNEY, 1947). Para comparações entre múltiplos grupos, foi utilizado o teste de Kruskal-Wallis (KRUSKAL; WALLIS, 1952). Quando identificadas diferenças significativas entre múltiplos cenários, foram aplicados testes *pós-hoc* de Dunn com correção de Bonferroni (DUNN, 1964). Além dos testes de significância, foi utilizado o tamanho de efeito de Cliff's Delta para avaliar a magnitude das diferenças entre distribuições (CLIFF, 1993).

A etapa de modelagem quantitativa será conduzida a partir da integração entre os achados da literatura, os resultados experimentais e características informacionais dos dados. A proposta não busca substituir a experimentação, mas oferecer uma forma de apoio preliminar à análise de alternativas arquiteturais. A intenção é investigar se características como volume, taxa de eventos, entropia, redundância e variabilidade podem ser relacionadas a métricas de desempenho, contribuindo para uma tomada de decisão arquitetural mais fundamentada.

3.4 Validade da Pesquisa

A validade da pesquisa será analisada considerando possíveis ameaças à validade interna, externa, de construção e de conclusão. Essa organização é comum em estudos experimentais em Engenharia de Software e contribui para tornar explícitas as limitações do desenho experimental, da coleta de dados e da interpretação dos resultados (WOHLIN et al., 2012).

A validade interna está relacionada à capacidade de atribuir os efeitos observados às variáveis controladas no experimento. Para mitigar ameaças internas, os cenários foram definidos previamente, as execuções foram automatizadas e as métricas foram coletadas de forma sistemática. Além disso, cada cenário foi executado repetidas vezes, reduzindo a influência de variações ocasionais do ambiente.

A validade externa refere-se à possibilidade de generalização dos resultados para outros contextos. Como o experimento utiliza um ambiente específico, baseado em Spark, Kafka e um *dataset* definido, os resultados devem ser interpretados dentro desse contexto. A ampliação para outros *datasets*, tecnologias e ambientes de execução será indicada como possibilidade de trabalhos futuros.

A validade de construção está relacionada à adequação das métricas utilizadas para representar os conceitos investigados. Para mitigar essa ameaça, foram utilizadas métricas amplamente adotadas em avaliações de desempenho, como *throughput*, latência, uso de recursos, *backpressure* e escalabilidade.

A validade de conclusão diz respeito à adequação dos procedimentos estatísticos utilizados para interpretar os resultados. Para reduzir essa ameaça, foram aplicados testes de normali-

dade, testes não paramétricos, pós-testes e medidas de tamanho de efeito, evitando conclusões baseadas apenas em médias ou observações visuais.

Dessa forma, a metodologia proposta permite que a pesquisa avance de uma compreensão ampla da literatura para uma avaliação empírica específica e, posteriormente, para uma proposta de modelagem quantitativa voltada ao apoio à decisão arquitetural em sistemas intensivos em dados.

4

Revisão Sistemática da Literatura

A Revisão Sistemática da Literatura (RSL) constituiu a primeira etapa da pesquisa e teve como objetivo consolidar evidências empíricas sobre estratégias de *design*, técnicas arquiteturais, métricas de avaliação e lacunas relacionadas a sistemas intensivos em dados. Essa etapa fornece a base conceitual e empírica para a campanha experimental e para a proposta de um modelo de apoio à decisão arquitetural.

A RSL resultou no artigo intitulado *Design of Data-Intensive Systems: A Systematic Review with Exploratory Quantitative Synthesis*, submetido ao WEBIST 2026. No momento da elaboração desta proposta, o artigo encontra-se aguardando aprovação. Essa submissão representa um dos resultados parciais desta pesquisa e consolida a etapa de revisão sistemática desenvolvida no âmbito da dissertação.

A condução da revisão seguiu diretrizes reconhecidas para Revisões Sistemáticas da Literatura em Engenharia de Software (KITCHENHAM; CHARTERS, 2007; KITCHENHAM et al., 2009; KITCHENHAM; BUDGEN; BRERETON, 2015) e utilizou o protocolo PRISMA 2020 como referência para relatar o processo de identificação, triagem, elegibilidade e inclusão dos estudos (PAGE et al., 2021). Além da síntese qualitativa, foi conduzida uma síntese quantitativa exploratória com o objetivo de comparar, de forma normalizada, os efeitos de diferentes otimizações arquiteturais reportadas na literatura.

4.1 Objetivo e Questões de Pesquisa

O objetivo da RSL foi identificar, organizar e sintetizar evidências sobre o *design* de sistemas intensivos em dados, com ênfase em estratégias arquiteturais orientadas ao desempenho. A revisão buscou responder às seguintes questões de pesquisa:

1. Quais técnicas e abordagens de *design* são aplicadas em sistemas intensivos em dados?

2. Quais métricas são utilizadas para avaliar o desempenho e a eficiência desses sistemas?
3. Quais evidências quantitativas existem sobre a efetividade das diferentes abordagens?
4. Quais estilos arquiteturais, *frameworks* ou padrões de *design* são mais recorrentes em sistemas intensivos em dados?
5. Existem lacunas ou oportunidades para estudos futuros?

Essas questões permitiram orientar a busca, a seleção, a extração dos dados e a análise dos estudos primários. A definição das questões também contribuiu para delimitar o foco da revisão, evitando a inclusão de estudos genéricos sobre *Big Data* que não apresentassem relação direta com *design* arquitetural, métricas de desempenho ou avaliação empírica.

4.2 Estratégia de Busca

A busca foi realizada em quatro bibliotecas digitais: ACM Digital Library¹, IEEE Xplore², Scopus³ e ScienceDirect⁴. Essas bases foram selecionadas por sua relevância para as áreas de Engenharia de Software, Sistemas Distribuídos, Arquitetura de Software e Computação Intensiva em Dados.

A estratégia de busca foi construída com base na estrutura PICO, considerando quatro dimensões principais: população, intervenção, comparação e resultado (*outcome*). A população foi representada por termos associados a sistemas intensivos em dados, como *data-intensive systems*, *big data systems* e *large-scale data systems*. A intervenção foi representada por termos relacionados a *design*, arquitetura e projeto arquitetural. A comparação incluiu termos associados a atributos de qualidade, como desempenho, escalabilidade, confiabilidade e tolerância a falhas. O resultado foi representado por termos relacionados a métricas e avaliação quantitativa.

De forma geral, a *string* de busca combinou os termos por meio de operadores booleanos, conforme a lógica apresentada a seguir:

$$S = (P) \wedge (I) \wedge (C) \wedge (O) \quad (4.1)$$

em que *P* representa os termos associados à população, *I* representa os termos associados à intervenção, *C* representa os termos associados aos atributos comparativos e *O* representa os termos associados aos resultados esperados. Essa formulação permitiu estruturar a busca de modo sistemático e alinhado ao objetivo da revisão.

¹<<https://dl.acm.org/>>

²<<https://ieeexplore.ieee.org>>

³<<https://www.scopus.com>>

⁴<<https://www.sciencedirect.com/>>

A busca foi conduzida em setembro de 2025. Foram considerados estudos publicados em periódicos revisados por pares ou em anais de conferências, desde que apresentassem relação direta com sistemas intensivos em dados, *design* arquitetural, avaliação de desempenho ou otimizações sistêmicas.

4.3 Critérios de Inclusão e Exclusão

Para assegurar a relevância e a qualidade dos estudos selecionados, foram definidos critérios de inclusão e exclusão antes da execução da triagem. Esses critérios foram aplicados nas etapas de leitura de títulos, resumos e textos completos.

Tabela 2 – Critérios de inclusão e exclusão da RSL

Critérios de Inclusão	Critérios de Exclusão
Estudos publicados em periódicos revisados por pares ou conferências.	Estudos puramente conceituais, sem resultados empíricos.
Estudos sobre <i>design</i> , arquitetura ou implementação de sistemas intensivos em dados.	Trabalhos sem métricas quantitativas ou sem descrição do ambiente experimental.
Trabalhos que comparam a solução proposta com pelo menos uma configuração de referência (<i>baseline</i>).	Estudos não escritos em inglês.
Pesquisas com resultados quantitativos, como desempenho, escalabilidade ou consumo de recursos.	Trabalhos sobre sistemas de dados em geral, sem foco específico em sistemas intensivos em dados ou <i>design</i> arquitetural.
Estudos com texto completo disponível e documentação mínima do ambiente experimental.	Revisões de literatura, meta-análises, cartas, editoriais ou capítulos de livro.

Fonte: elaborado pelo autor.

Nos casos em que houve dúvidas sobre a aplicação dos critérios, a decisão foi discutida entre os revisores. Quando não foi possível alcançar consenso, um avaliador independente foi consultado. As razões de exclusão foram registradas ao longo do processo, garantindo maior transparência e rastreabilidade à revisão.

4.4 Processo de Seleção dos Estudos

O processo de busca nas quatro bibliotecas digitais identificou 979 registros. A distribuição inicial foi de 448 artigos na ACM Digital Library, 215 na IEEE Xplore, 35 na ScienceDirect e 281 na Scopus. Após a remoção de 100 duplicatas, restaram 879 estudos únicos para a etapa de triagem.

A triagem por títulos e resumos resultou na exclusão de 600 registros que não atendiam aos critérios estabelecidos. Em seguida, 279 textos completos foram avaliados. Nessa etapa, 269 estudos foram excluídos, principalmente por não apresentarem resultados quantitativos

compatíveis com o foco da síntese, por não descreverem adequadamente o ambiente experimental ou por não permitirem comparação com uma configuração de referência.

Ao final do processo, 10 estudos atenderam a todos os critérios de elegibilidade e qualidade, sendo incluídos tanto na Revisão Sistemática da Literatura quanto na síntese quantitativa exploratória.

A Figura 3 apresenta o fluxo de seleção dos estudos de acordo com o modelo PRISMA 2020.

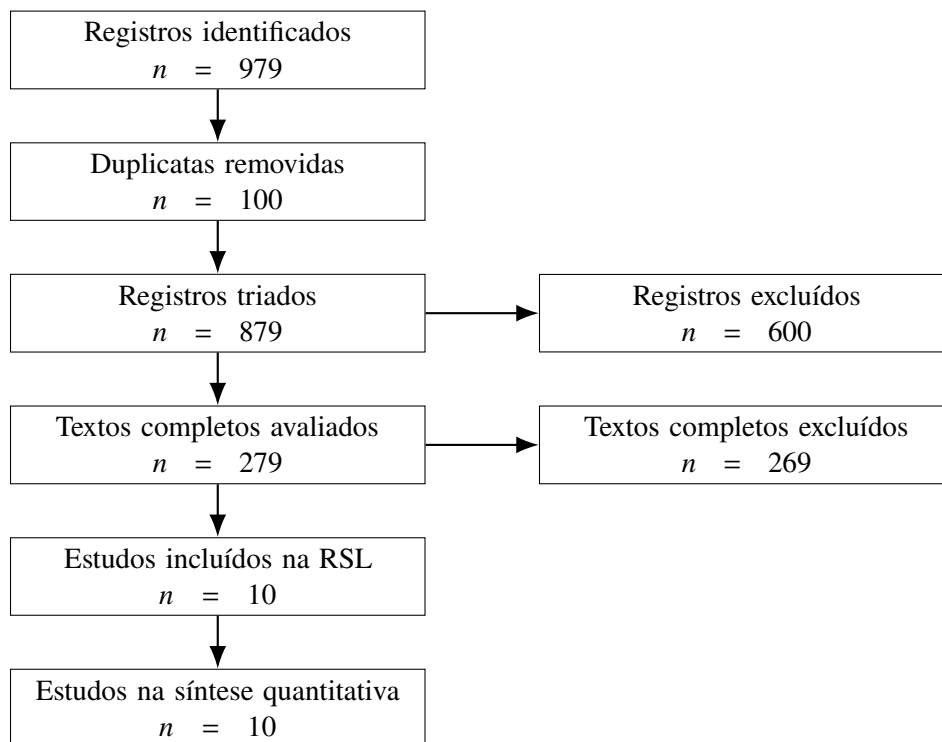


Figura 3 – Fluxo PRISMA 2020 do processo de seleção dos estudos

A Tabela 3 apresenta a distribuição dos registros identificados por base de dados.

Tabela 3 – Distribuição dos registros identificados por base de dados

Base de dados	Registros identificados
ACM Digital Library	448
IEEE Xplore	215
ScienceDirect	35
Scopus	281
Total	979

Fonte: elaborado pelo autor.

4.5 Avaliação da Qualidade

A avaliação da qualidade dos estudos foi conduzida com base em um *checklist* composto por 14 critérios. Esses critérios avaliaram aspectos como clareza do problema, aderência ao tema, descrição do contexto, metodologia, uso de dados experimentais, métricas arquiteturais, replicabilidade, limitações, recomendações práticas, apresentação de valores numéricos, comparabilidade das métricas, descrição do ambiente experimental, medidas de dispersão e comparação entre técnicas ou arquiteturas.

Cada critério recebeu uma das seguintes pontuações: Sim = 1, Parcialmente = 0,5 e Não = 0. Considerando os 14 critérios, a pontuação máxima possível para cada estudo foi de 14 pontos. Foi adotado um ponto de corte de 8 pontos, correspondente a aproximadamente 60% da pontuação máxima, conforme representado pela Equação 4.2.

$$QA_{min} = 0.60 \times 14 = 8.4 \approx 8 \quad (4.2)$$

Apenas estudos que atingiram ou superaram esse limiar foram mantidos no conjunto final. A adoção desse critério buscou garantir que os estudos incluídos apresentassem qualidade metodológica mínima para sustentar a síntese qualitativa e a síntese quantitativa exploratória.

Tabela 4 – Dimensões avaliadas no checklist de qualidade

Dimensão	Aspectos avaliados
Clareza do estudo	Definição do problema, objetivo e aderência ao <i>design</i> de sistemas intensivos em dados.
Contexto experimental	Descrição do sistema, volume de dados, ambiente de execução, <i>hardware</i> , <i>software</i> e <i>workload</i> .
Métricas e resultados	Uso de métricas quantitativas, apresentação de valores numéricos e comparabilidade dos resultados.
Rigor metodológico	Metodologia clara, replicabilidade, uso de <i>baseline</i> e apresentação de limitações.
Relevância prática	Diretrizes, recomendações ou implicações para o <i>design</i> de sistemas intensivos em dados.

Fonte: elaborado pelo autor.

4.6 Extração dos Dados

Os dados foram extraídos por meio de um formulário padronizado, implementado em uma planilha previamente definida. Para cada estudo incluído, foram registrados os seguintes campos: identificação do estudo, autores, ano de publicação, veículo de publicação, base digital, tipo de sistema investigado, domínio de aplicação, técnica arquitetural ou otimização proposta, métricas utilizadas e resultados quantitativos reportados.

Também foram extraídas informações relacionadas ao ambiente experimental, incluindo *hardware*, *software*, *datasets*, *workloads*, número de execuções, médias, desvios-padrão, variâncias e demais medidas estatísticas, quando disponíveis. A extração foi conduzida por um revisor principal e verificada por um segundo revisor. Divergências foram resolvidas por discussão e, quando necessário, por consulta a um terceiro avaliador.

Essa etapa foi essencial para permitir tanto a síntese qualitativa quanto a síntese quantitativa exploratória, uma vez que os estudos selecionados apresentavam métricas heterogêneas e diferentes formas de relato experimental.

4.7 Síntese Quantitativa Exploratória

Em vez de realizar uma meta-análise inferencial formal, esta pesquisa adotou uma síntese quantitativa exploratória. Essa decisão foi motivada pela ausência sistemática de medidas de variância, desvio-padrão e tamanho amostral em grande parte dos estudos primários. Diante dessa limitação, a ponderação individual dos estudos e a modelagem formal de efeitos aleatórios não seriam estatisticamente defensáveis.

Para permitir comparações entre estudos que utilizaram métricas heterogêneas, foi adotada a Razão de Resposta Logarítmica, ou *Log Response Ratio* (LRR), como medida normalizada de efeito. De forma geral, o LRR pode ser definido como:

$$LRR_i = \ln(RR_i) \quad (4.3)$$

em que RR_i representa a razão de resposta do estudo i . Quando a métrica analisada indicava que valores maiores eram melhores, como *throughput* ou *speedup*, a razão foi calculada conforme a Equação 4.4.

$$RR_i = \frac{M_{tratamento}}{M_{baseline}} \quad (4.4)$$

Quando a métrica indicava que valores menores eram melhores, como tempo de execução, latência ou tempo de recuperação, a razão foi invertida, conforme a Equação 4.5.

$$RR_i = \frac{M_{baseline}}{M_{tratamento}} \quad (4.5)$$

A média não ponderada dos valores de LRR foi utilizada para estimar a magnitude geral dos efeitos observados:

$$\overline{LRR} = \frac{1}{k} \sum_{i=1}^k LRR_i \quad (4.6)$$

em que k representa o número de estudos incluídos na síntese quantitativa. Para facilitar a interpretação, o LRR médio também pode ser convertido novamente para a escala de razão de resposta:

$$RR_{medio} = e^{\overline{LRR}} \quad (4.7)$$

Como o valor médio observado foi $\overline{LRR} = 0.981$, a razão média de resposta foi aproximadamente:

$$RR_{medio} = e^{0.981} \approx 2.7 \quad (4.8)$$

Esse resultado indica que, de forma exploratória, as otimizações analisadas apresentaram impacto médio positivo no desempenho, com ganhos aproximados de 2,7 vezes em relação às configurações de referência. No entanto, esse valor deve ser interpretado com cautela, pois a síntese não utiliza ponderação por variância ou tamanho amostral.

4.8 Estudos Incluídos e Resultados

Os dez estudos incluídos na síntese quantitativa exploratória investigam diferentes tipos de otimizações em sistemas intensivos em dados, abrangendo desde técnicas de escalonamento e compilação SQL até estratégias de armazenamento, processamento próximo aos dados e tolerância a falhas (WANG et al., 2018; AUNG; MIN; MAW, 2019; ALQUWAIEE; WU, 2022; PONCE et al., 2019; LI et al., 2022; VINÇON et al., 2020; LYU et al., 2022; HAYNES; CHEUNG; BALAZINSKA, 2016; SCHIAVIO; BONETTA; BINDER, 2020; GÖTZE; BAUMANN; SATTTLER, 2018).

A Tabela 5 apresenta uma síntese dos estudos incluídos, suas otimizações principais, métricas utilizadas e valores de LRR.

Os resultados indicam que as maiores magnitudes de efeito estão associadas a intervenções em baixo nível, especialmente aquelas relacionadas a armazenamento, movimentação de dados e execução de consultas. Em contraste, otimizações em camadas de controle, como escalonamento e tolerância a falhas, apresentaram ganhos mais moderados, embora ainda relevantes do ponto de vista arquitetural.

4.9 Distribuição Temática dos Estudos

A análise dos estudos incluídos permitiu agrupá-los em três categorias temáticas principais: otimização de consultas e execução, otimização de hardware e armazenamento, e integração e conectividade. A primeira categoria correspondeu a 40% dos estudos, enquanto as duas demais representaram 30% cada.

Tabela 5 – Síntese dos estudos primários incluídos na RSL

Estudo	Otimização principal	Métrica analisada	LRR
Wang et al. (2018)	Escalonamento em Spark/YARN	Taxa de conclusão	0,233
Aung et al. (2019)	Tolerância a falhas com <i>checkpoint</i>	Tempo de recuperação	0,357
AlQuwaiee et al. (2022)	Modelagem de desempenho em Spark	Tempo de execução	0,511
Ponce et al. (2019)	Integração entre HPC e <i>Big Data</i>	<i>Throughput</i>	0,631
Li et al. (2022)	Protocolo transacional escalável	<i>Throughput</i>	0,693
Vinçon et al. (2020)	Processamento próximo aos dados	Desempenho	0,993
Lyu et al. (2022)	Gerenciamento fino de recursos	Latência/custo	1,273
Haynes et al. (2016)	Geração de <i>pipelines</i> de dados	<i>Speedup</i>	1,335
Schiavio et al. (2020)	Otimização dinâmica de compilação	<i>Speedup</i>	1,482
Götze et al. (2018)	Layout de armazenamento para NVM	Desempenho	2,303

Fonte: elaborado pelo autor.

A Figura 4 apresenta a distribuição temática dos estudos incluídos.

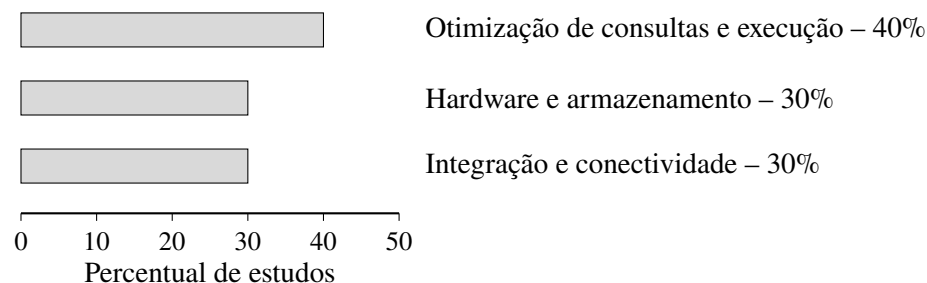


Figura 4 – Distribuição temática dos estudos incluídos

A categoria de otimização de consultas e execução inclui estudos relacionados ao escalonamento, à compilação SQL e ao gerenciamento de recursos. A categoria de hardware e armazenamento concentra estudos sobre memória não volátil, processamento próximo aos dados e protocolos transacionais escaláveis. A categoria de integração e conectividade reúne estudos voltados à eficiência na movimentação de dados, integração entre ambientes e tolerância a falhas.

Essa distribuição evidencia que parte significativa das contribuições da literatura está concentrada em camadas de execução e infraestrutura. Esse resultado reforça a importância de decisões arquiteturais que considerem não apenas a organização lógica do sistema, mas também aspectos de baixo nível, como armazenamento, movimentação de dados, execução de consultas

e gerenciamento de recursos.

4.10 Lacunas Identificadas

A RSL permitiu identificar lacunas importantes na literatura sobre *design* de sistemas intensivos em dados. A primeira lacuna refere-se à ausência de padronização no relato experimental. Muitos estudos reportam métricas como tempo de execução, latência, *throughput* e *speedup*, mas nem sempre apresentam informações suficientes sobre o número de execuções, medidas de dispersão, intervalos de confiança ou detalhes completos do ambiente experimental. Essa limitação dificulta a replicação dos estudos e restringe a realização de meta-análises inferenciais.

A segunda lacuna está relacionada à heterogeneidade das métricas e dos cenários avaliados. Embora os estudos investiguem sistemas intensivos em dados, as técnicas, *workloads*, *datasets*, ambientes e métricas variam significativamente. Essa diversidade reforça a necessidade de medidas normalizadas, como o LRR, mas também exige cautela na interpretação dos resultados agregados.

A terceira lacuna refere-se à concentração dos estudos em otimizações específicas. Muitos trabalhos analisam técnicas pontuais em ambientes controlados, mas poucos oferecem diretrizes gerais para apoiar decisões arquiteturais em contextos variados. Como consequência, ainda há necessidade de abordagens que auxiliem arquitetos de software a relacionar características dos dados, estratégias de processamento e métricas de desempenho.

A quarta lacuna envolve a limitada integração entre características dos dados e decisões arquiteturais. A maioria dos estudos avalia o desempenho a partir de métricas operacionais, mas poucos investigam como propriedades dos dados, como distribuição, cardinalidade, redundância, variabilidade ou entropia, podem influenciar a escolha entre alternativas arquiteturais. Essa lacuna sustenta a proposta desta dissertação de investigar uma abordagem quantitativa de apoio à decisão arquitetural baseada em evidências empíricas e características observáveis dos dados.

A partir dessas lacunas, a RSL fornece a base metodológica e empírica para as próximas etapas da pesquisa. Seus resultados justificam a condução da campanha experimental apresentada no Capítulo 5 e fundamentam a proposta de um modelo quantitativo de apoio à decisão arquitetural em sistemas intensivos em dados.

5

Campanha Experimental

A campanha experimental constituiu a segunda etapa da pesquisa e teve como objetivo avaliar empiricamente o comportamento de estratégias de processamento *batch* e *streaming* em um cenário de sistema intensivo em dados. Para isso, foi construído um ambiente experimental baseado em Apache Spark, Spark Structured Streaming, Apache Kafka e contêineres Docker, utilizando o *dataset* NYC Taxi como fonte de dados.

A condução da campanha experimental foi motivada pelas lacunas identificadas na Revisão Sistemática da Literatura, especialmente a necessidade de evidências empíricas mais controladas, comparáveis e estatisticamente analisáveis. A partir dessa etapa, buscou-se observar como diferentes decisões arquiteturais e configurações operacionais afetam métricas como *throughput*, latência, escalabilidade, consumo de recursos e estabilidade do processamento.

Nesta pesquisa, a campanha experimental é tratada como um experimento controlado principal, composto por múltiplos cenários experimentais. Essa organização permite avaliar diferentes fatores — como volume de dados, taxa de eventos, número de *workers* e intervalo de *trigger* — sem fragmentar a pesquisa em um conjunto de experimentos independentes. O desenho experimental foi orientado por princípios de experimentação em Engenharia de Software, considerando planejamento, definição de variáveis, controle de fatores, repetição das execuções e análise estatística dos resultados (WOHLIN et al., 2012).

5.1 Objetivo e Questões Experimentais

O objetivo da campanha experimental foi comparar, de forma empírica e reproduzível, o comportamento dos paradigmas de processamento *batch* e *streaming* sob diferentes condições de carga, paralelismo e configuração temporal. O foco recaiu sobre a análise de como decisões arquiteturais relacionadas ao paradigma de processamento, ao volume dos dados, à taxa de eventos, ao número de *workers* e à cadência dos micro-lotes impactam o desempenho de um

pipeline intensivo em dados.

De modo geral, a campanha buscou responder à seguinte questão norteadora: *como diferentes estratégias de processamento de dados se comportam em termos de throughput, latência, escalabilidade e consumo de recursos em um ambiente distribuído baseado em Spark e Kafka?*

A partir dessa questão geral, foram definidas quatro questões experimentais específicas:

1. **RQ1:** Qual paradigma apresenta maior *throughput* em sistemas intensivos em dados: processamento *batch* ou processamento *streaming*?
2. **RQ2:** Como o volume de dados afeta o desempenho dos paradigmas *batch* e *streaming*?
3. **RQ3:** Como a taxa de eventos e a cadência de micro-lotes afetam a latência e a estabilidade do processamento *streaming*?
4. **RQ4:** Como a escalabilidade horizontal afeta o desempenho dos paradigmas *batch* e *streaming*?

A RQ1 compara diretamente os paradigmas de processamento. A RQ2 investiga o impacto do volume de dados sobre o desempenho de cada paradigma. A RQ3 analisa aspectos específicos do processamento em fluxo contínuo, especialmente a relação entre taxa de eventos e intervalo de *trigger*. Por fim, a RQ4 investiga o efeito da escalabilidade horizontal — medida pelo número de *workers* Spark — sobre o desempenho de ambos os paradigmas.

5.2 Ambiente Experimental

O ambiente experimental foi implementado utilizando contêineres Docker, com o objetivo de isolar e controlar os componentes do *pipeline*. A arquitetura experimental incluiu um *cluster* Spark, um serviço Kafka, *scripts* de geração e envio de eventos, *jobs* de processamento *batch* e *streaming*, além de *scripts* de monitoramento e consolidação dos resultados. Todos os experimentos foram executados em uma máquina com processador AMD Ryzen 7 7435HS (8 núcleos físicos e 16 *threads*), 61,53 GB de memória RAM, 467,35 GB de armazenamento interno e sistema operacional Linux (kernel 7.0.0-15-generic).

O Apache Spark foi utilizado como mecanismo de processamento distribuído. Nos cenários *batch*, o Spark processou amostras de dados previamente armazenadas em disco. Nos cenários *streaming*, o Spark Structured Streaming consumiu eventos publicados em tópicos Kafka e os processou continuamente por meio de micro-lotes (ARMBRUST et al., 2018; ZAHARIA, 2016).

O Apache Kafka atuou como sistema de mensageria distribuído e camada de ingestão dos eventos. Sua adoção permitiu simular um fluxo contínuo de dados, com produtores enviando

registros em taxas controladas e consumidores processando os eventos por meio do Spark Structured Streaming (KREPS; NARKHEDE; RAO, 2011).

A Figura 5 apresenta uma visão geral da arquitetura experimental adotada.

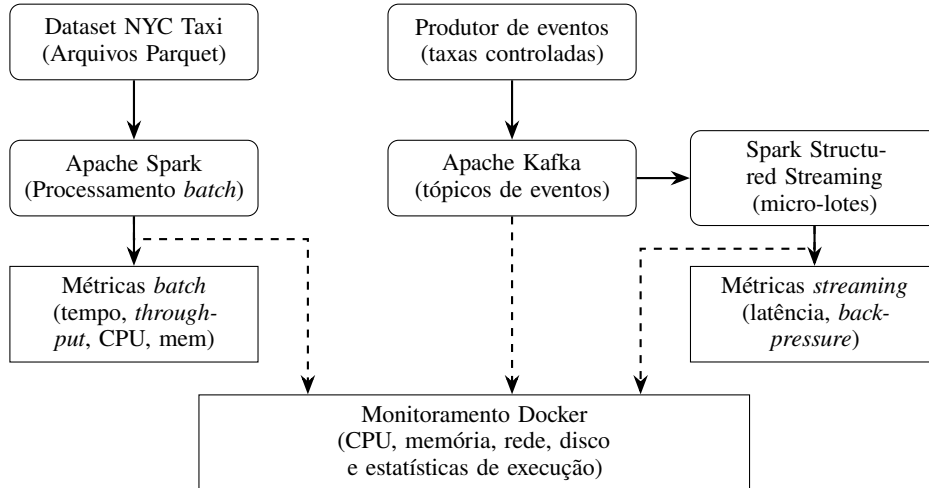


Figura 5 – Arquitetura geral da campanha experimental organizada em camadas funcionais.

A Figura 5 detalha a arquitetura em camadas adotada na campanha experimental, dividida em três pilares principais: ingestão, processamento e telemetria. Na camada superior, os dados históricos do Dataset NYC Taxi alimentam o ecossistema batch via Apache Spark, enquanto o Produtor de Eventos simula a chegada contínua de dados em tempo real com taxas controladas para o Apache Kafka. No núcleo do processamento, o Apache Spark manipula os dados em lote e o Spark Structured Streaming consome os tópicos do Kafka em micro-lotes, gerando as respectivas métricas de desempenho e resiliência (throughput, latência e backpressure). Por fim, integrado na base da infraestrutura, o módulo de Monitoramento Docker atua de forma transversal por meio de conexões de telemetria (representadas pelas linhas tracejadas), coletando continuamente o consumo de recursos físicos cruciais como CPU, memória, rede e disco durante a execução de ambos os fluxos.

5.3 Dataset e Preparação dos Dados

O *dataset* utilizado na campanha experimental foi o NYC Taxi, amplamente empregado em estudos envolvendo processamento de dados em larga escala, análise de mobilidade urbana e *workloads* analíticos distribuídos. Os dados foram utilizados no formato Parquet e organizados em amostras de diferentes tamanhos, permitindo avaliar o impacto do volume sobre o desempenho dos cenários experimentais.

Foram consideradas quatro escalas de dados: 200 MB, 1 GB, 3 GB e 10 GB. A utilização de volumes distintos teve como objetivo verificar se o aumento do tamanho do *dataset* impacta de maneira semelhante os paradigmas *batch* e *streaming*, bem como avaliar se *workloads* maiores favorecem a amortização de *overheads* distribuídos.

Nos cenários *batch*, os arquivos Parquet foram lidos e processados diretamente pelo Spark a partir do sistema de arquivos local. Nos cenários *streaming*, os dados foram convertidos em eventos discretos e enviados ao Kafka por meio de *scripts* produtores, respeitando taxas de emissão previamente definidas. Essa estratégia permitiu simular um fluxo contínuo de dados e controlar a pressão exercida sobre o *pipeline* de *streaming*.

5.4 Variáveis e Métricas

A campanha experimental foi estruturada a partir de variáveis independentes e dependentes. As variáveis independentes representam os fatores controlados durante a execução dos cenários; as variáveis dependentes correspondem às métricas coletadas para avaliar o comportamento do sistema.

A Tabela 6 apresenta as variáveis independentes consideradas, com seus respectivos valores.

Tabela 6 – Variáveis independentes da campanha experimental

Variável independente	Valores considerados
Paradigma de processamento	Processamento <i>batch</i> e processamento <i>streaming</i>
Volume de dados	200 MB, 1 GB, 3 GB e 10 GB
Taxa de eventos	200, 500, 1.000 e 2.000 eventos por segundo
Número de <i>workers</i> Spark	1, 2 e 4 <i>workers</i>
Intervalo de <i>trigger</i>	1 s, 2 s e 5 s

Fonte: elaborado pelo autor.

Essas variáveis representam decisões ou configurações arquiteturais relevantes em *pipelines* de dados. A escolha do paradigma define a forma geral de processamento; o volume e a taxa de eventos representam características da carga de trabalho; o número de *workers* define o nível de paralelismo disponível; e o intervalo de *trigger* determina a cadência de formação dos micro-lotes no Spark Structured Streaming.

A Tabela 7 apresenta as principais métricas coletadas durante a campanha.

O *throughput* foi utilizado como métrica primária para comparar a capacidade de processamento entre os cenários. Formalmente, essa grandeza é expressa pela Equação 5.1:

$$\text{Throughput} = \frac{N_{\text{registros}}}{T_{\text{processamento}}} \quad (5.1)$$

em que $N_{\text{registros}}$ representa a quantidade de registros processados e $T_{\text{processamento}}$ o tempo total de processamento.

Tabela 7 – Métricas avaliadas na campanha experimental

Métrica	Descrição
Tempo total de execução	Tempo decorrido entre o início e a conclusão do processamento em cada cenário.
<i>Throughput</i>	Quantidade de registros ou eventos processados por unidade de tempo.
Latência	Intervalo de tempo entre a chegada de um evento e a conclusão de seu processamento, especialmente relevante nos cenários <i>streaming</i> .
Uso de CPU	Utilização média e máxima da CPU nos contêineres durante a execução.
Consumo de memória	Uso médio e máximo de memória nos contêineres.
<i>Backpressure</i>	Indicador de pressão no <i>pipeline</i> quando a taxa de chegada dos eventos supera a capacidade de processamento.
Taxa de processamento	Métrica do Spark Structured Streaming que expressa o número de registros processados por segundo em cada micro-lote.
Rede e disco	Tráfego de entrada e saída dos contêineres durante a execução dos cenários.

Fonte: elaborado pelo autor.

Nos cenários de *streaming*, a latência foi interpretada como o intervalo de tempo entre a produção de um evento e a conclusão de seu processamento pelo Spark Structured Streaming. Essa relação é representada pela Equação 5.2:

$$\text{Latência} = T_{\text{processamento_evento}} - T_{\text{chegada_evento}} \quad (5.2)$$

em que $T_{\text{processamento_evento}}$ é o instante de conclusão do processamento e $T_{\text{chegada_evento}}$ é o instante de chegada do evento ao *pipeline*.

Essas métricas foram selecionadas porque representam dimensões complementares do desempenho de sistemas intensivos em dados. Enquanto *throughput* e latência expressam a capacidade de processamento e o tempo de resposta do sistema, as métricas de CPU, memória, rede e disco permitem quantificar o custo computacional associado a cada configuração experimental.

5.5 Cenários Experimentais

Os cenários experimentais foram organizados em blocos temáticos, permitindo isolar diferentes fatores e observar seus efeitos individuais sobre o desempenho do sistema. Ao todo, fo-

ram definidos 35 cenários, sendo 16 relacionados ao processamento *batch* e 19 ao processamento *streaming*.

A Tabela 8 apresenta a organização geral dos cenários, seus objetivos e configurações principais.

Tabela 8 – Organização dos cenários experimentais

Grupo	Objetivo	Configurações principais	Qtd.
BL1–BL4	Avaliar o impacto do volume em <i>batch</i>	200 MB, 1 GB, 3 GB e 10 GB; 1 <i>worker</i>	4
BS1–BS12	Avaliar escalabilidade horizontal em <i>batch</i>	1, 2 e 4 <i>workers</i> para cada volume de dados	12
SL1–SL4	Avaliar o impacto da taxa em <i>streaming</i>	200, 500, 1.000 e 2.000 eventos/s; <i>trigger</i> de 1 s; 1 <i>worker</i>	4
SS1–SS12	Avaliar escalabilidade horizontal em <i>streaming</i>	1, 2 e 4 <i>workers</i> para cada taxa de eventos	12
ST1–ST3	Avaliar sensibilidade ao <i>trigger</i>	1.000 eventos/s; 1 <i>worker</i> ; <i>triggers</i> de 1 s, 2 s e 5 s	3
Total	—	—	35

Fonte: elaborado pelo autor.

Os cenários *batch* foram organizados em dois blocos. O primeiro (BL1–BL4) avaliou o impacto do volume de dados com número fixo de *workers*. O segundo (BS1–BS12) avaliou a escalabilidade horizontal, variando o número de *workers* para cada volume de dados. Essa estrutura permitiu verificar se o aumento do paralelismo produz ganhos consistentes ou se introduz custos de coordenação que limitam a eficiência esperada.

Os cenários *streaming* foram divididos em três blocos. O primeiro (SL1–SL4) avaliou o impacto da taxa de eventos com *trigger* fixo e um único *worker*. O segundo (SS1–SS12) avaliou a escalabilidade horizontal variando o número de *workers*. O terceiro (ST1–ST3) avaliou a sensibilidade do *pipeline* de *streaming* a diferentes intervalos de *trigger*. Essa estrutura possibilitou analisar tanto a capacidade de absorção de eventos quanto a estabilidade temporal do processamento em fluxo contínuo.

5.6 Execução e Repetições

Cada cenário foi executado 30 vezes. Considerando os 35 cenários definidos, a campanha experimental totalizou 1.050 execuções, conforme indicado na Equação 5.3:

$$N_{\text{execuções}} = 35 \times 30 = 1.050 \quad (5.3)$$

A repetição dos cenários teve como objetivo reduzir a influência de variações aleatórias do ambiente e permitir uma análise estatística mais robusta das métricas coletadas. Antes da

campanha completa, foi realizada uma etapa de validação operacional com uma execução por cenário. Essa validação verificou o funcionamento correto de todos os componentes do *pipeline*, incluindo a infraestrutura Docker, o Kafka, o Spark, o produtor de eventos, a coleta de métricas, a geração de arquivos CSV e a consolidação dos resultados.

Após a validação operacional, a campanha completa foi executada com coleta automatizada das métricas. Os dados brutos foram armazenados em arquivos CSV e posteriormente consolidados para análise estatística e geração de relatórios.

5.7 Automação e Coleta de Métricas

A execução experimental foi integralmente automatizada por meio de *scripts* responsáveis por preparar o ambiente, gerar amostras de dados, executar os *jobs* de processamento, monitorar os contêineres e consolidar os resultados. A automação teve como objetivos reduzir interferências manuais, aumentar a reprodutibilidade e padronizar a coleta das métricas¹.

Os artefatos produzidos incluem resultados brutos das execuções *batch* e *streaming*, séries temporais de métricas dos contêineres, estatísticas de micro-lotes, sumários por cenário, gráficos e relatórios textuais. Esses artefatos permitiram analisar tanto o comportamento agregado dos cenários quanto variações específicas observadas durante a execução dos *jobs*.

A coleta de métricas dos contêineres permitiu monitorar CPU, memória, rede e disco ao longo de toda a execução. Nos cenários de *streaming*, foram coletadas adicionalmente métricas específicas do Spark Structured Streaming, como taxa de processamento por micro-lote, latência de *trigger*, latência de eventos e *backpressure*.

5.8 Procedimento de Análise Estatística

A análise dos resultados foi conduzida em etapas sequenciais. Inicialmente, foram calculadas estatísticas descritivas por cenário, incluindo média, mediana, desvio padrão e coeficiente de variação. Em seguida, foi aplicado o teste de Shapiro-Wilk para avaliar a normalidade das distribuições (SHAPIRO; WILK, 1965).

Como métricas de desempenho em sistemas computacionais não devem ser assumidas como normalmente distribuídas, foram adotados testes estatísticos não paramétricos. Para a comparação entre os paradigmas *batch* e *streaming*, foi aplicado o teste de Mann-Whitney U (MANN; WHITNEY, 1947). Para comparações entre múltiplos grupos — como diferentes volumes, taxas de eventos, intervalos de *trigger* e quantidades de *workers* — foi utilizado o teste de Kruskal-Wallis (KRUSKAL; WALLIS, 1952). Quando necessário, foram aplicados testes pós-hoc de Dunn com correção de Bonferroni (DUNN, 1964).

¹Os *scripts* de automação, os arquivos de configuração do Docker e os dados brutos coletados estão disponíveis publicamente em: <<https://github.com/jbruno97/stream-batch-experiment>>.

Além dos testes de significância, foi calculado o tamanho de efeito de Cliff's Delta para avaliar a magnitude das diferenças entre distribuições (CLIFF, 1993). Essa análise permitiu interpretar não apenas a existência de diferenças estatisticamente significativas, mas também sua relevância prática no contexto experimental.

A organização em cenários temáticos, a execução com 30 repetições por cenário, a coleta automatizada de métricas e o uso de testes não paramétricos fornecem uma base empírica rigorosa para discutir os *trade-offs* envolvidos na escolha entre estratégias de processamento em sistemas intensivos em dados. Os resultados obtidos a partir dessa campanha são discutidos nos capítulos seguintes, com ênfase no impacto das decisões arquiteturais sobre *throughput*, latência, escalabilidade e estabilidade do processamento.

6

Resultados Parciais

Os resultados parciais obtidos até o momento estão organizados em três blocos principais: os achados da Revisão Sistemática da Literatura (RSL), a síntese quantitativa exploratória dos estudos primários e os resultados da campanha experimental conduzida para comparar o processamento *batch* e o processamento *streaming*. Ao final, discutem-se as implicações desses resultados para a construção do modelo de apoio à decisão arquitetural.

Os resultados apresentados ainda devem ser interpretados como parciais, pois a pesquisa se encontra em estágio de desenvolvimento. No entanto, eles já permitem identificar tendências relevantes sobre o comportamento de sistemas intensivos em dados e sobre os fatores que influenciam decisões arquiteturais relacionadas a desempenho, escalabilidade, latência e estabilidade operacional.

6.1 Resultados da Revisão Sistemática da Literatura

A Revisão Sistemática da Literatura identificou inicialmente 979 registros nas quatro bibliotecas digitais selecionadas. Após a remoção de 100 duplicatas, restaram 879 registros únicos para triagem. A etapa de triagem por título, resumo e palavras-chave resultou na exclusão de 600 estudos. Em seguida, 279 textos completos foram avaliados quanto aos critérios de elegibilidade.

Ao final do processo, 10 estudos primários atenderam aos critérios de inclusão, exclusão e avaliação de qualidade. Esses estudos foram incluídos na síntese qualitativa e na síntese quantitativa exploratória. A distribuição inicial dos registros por biblioteca digital foi de 448 artigos na *ACM Digital Library*, 215 na *IEEE Xplore*, 35 na *ScienceDirect* e 281 na *Scopus*.

Esses resultados indicam que o tema possui presença relevante em bibliotecas voltadas à Computação, especialmente *ACM Digital Library*, *IEEE Xplore* e *Scopus*. Entretanto, o número final de estudos incluídos foi reduzido, evidenciando que poucos trabalhos atendiam simultane-

amente aos critérios de foco arquitetural, presença de avaliação empírica, comparação com um cenário de referência (*baseline*) e disponibilidade de dados quantitativos.

A análise qualitativa dos estudos primários permitiu agrupá-los em três categorias principais. A primeira categoria corresponde à otimização de consulta e execução, incluindo estudos relacionados a escalonamento, compilação SQL e gerenciamento de recursos em ambientes de Big Data (WANG et al., 2018; SCHIAVIO; BONETTA; BINDER, 2020; LYU et al., 2022). A segunda categoria envolve otimizações de armazenamento e *hardware*, contemplando estudos sobre NVM, processamento próximo aos dados (*near-data processing*) e arquiteturas escaláveis para ingestão transacional (GÖTZE; BAUMANN; SATTler, 2018; VINÇON et al., 2020; LI et al., 2022). A terceira categoria reúne estudos sobre integração, conectividade e confiabilidade, incluindo transferência de dados entre sistemas, integração HPC-Big Data e *mechanisms* de *checkpoint* em *pipelines* baseados em Apache Kafka (HAYNES; CHEUNG; BALAZINSKA, 2016; PONCE et al., 2019; AUNG; MIN; MAW, 2019).

Além dessas categorias, foi identificado um estudo voltado à modelagem e predição de desempenho em aplicações Spark-HBase, contribuindo diretamente para a discussão sobre mecanismos de apoio à configuração e tomada de decisão em sistemas de Big Data (ALQUWAIEE; WU, 2022).

A análise desses estudos mostra que os ganhos de desempenho em sistemas intensivos em dados podem ser obtidos por intervenções em diferentes camadas da arquitetura. Isso reforça a hipótese de que decisões arquiteturais não devem ser analisadas isoladamente, mas sim como parte de um conjunto de relações interdependentes entre dados, processamento, armazenamento, comunicação e recursos computacionais.

6.2 Resultados da Síntese Quantitativa Exploratória

A síntese quantitativa exploratória teve como objetivo estimar a magnitude dos efeitos reportados pelos estudos primários. Para isso, foi utilizado o *Log Response Ratio* (LRR) como medida normalizada de efeito. Essa escolha permitiu comparar resultados provenientes de diferentes métricas, como tempo de execução, latência, *throughput* e *speedup*.

De forma geral, os resultados indicaram efeitos positivos associados às técnicas avaliadas nos estudos primários. A síntese apontou que intervenções arquiteturais ou sistêmicas podem produzir melhorias relevantes de desempenho, especialmente quando atuam sobre gargalos de armazenamento, movimentação de dados, compilação de consultas e gerenciamento de recursos.

O valor médio da síntese exploratória foi de $\overline{\text{LRR}} = 0,981$, correspondente a uma razão média de resposta de aproximadamente $2,7\times$ em relação às configurações de referência. Esse resultado sugere que, nos estudos analisados, as otimizações arquiteturais produziram ganhos expressivos de desempenho. No entanto, essa interpretação deve ser feita com cautela, pois a

síntese foi baseada em média não ponderada e não em uma meta-análise inferencial formal.

A principal limitação observada na síntese quantitativa foi a ausência recorrente de informações completas sobre tamanho amostral, medidas de dispersão, intervalos de confiança e variância. Essa limitação impediu a aplicação de modelos estatísticos mais robustos, como modelos de efeitos aleatórios ponderados. Por esse motivo, a síntese foi tratada em caráter exploratório, com o objetivo de identificar tendências gerais e ordens de magnitude dos efeitos relatados.

Esse resultado é relevante para esta pesquisa porque reforça uma das lacunas centrais identificadas: embora existam evidências de ganhos de desempenho em sistemas intensivos em dados, essas evidências ainda se mostram fragmentadas, heterogêneas e de difícil comparação direta. Portanto, permanece necessária uma abordagem que combine evidências da literatura, dados experimentais controlados e características observáveis dos dados para apoiar decisões arquiteturais.

6.3 Resultados da Campanha Experimental

A segunda etapa da pesquisa consistiu em uma campanha experimental controlada para comparar o processamento *batch* e o processamento *streaming* utilizando Apache Spark, Spark Structured Streaming e Apache Kafka. Os resultados foram organizados e estruturados de acordo com as quatro questões de pesquisa experimentais definidas no estudo.

6.3.1 RQ1 — Paradigmas de Processamento e Vazão Máxima

Na primeira questão experimental (RQ1), foi avaliado qual paradigma apresenta maior *throughput* bruto em sistemas intensivos em dados. Os resultados demonstraram que o paradigma *batch* (mensurado em milhões de linhas por segundo) apresentou vazão massivamente superior ao *streaming* (mensurado em linhas por segundo) em todas as escalas de carga de trabalho testadas.

Para validar a significância das diferenças encontradas sem pressupostos de normalidade distributiva, aplicou-se o teste não-paramétrico de Mann-Whitney U de cauda dupla, complementado pelo cálculo do tamanho de efeito através do Delta de Cliff (δ). Conforme sumariado na Tabela 9, as divergências são estatisticamente significativas ($p < 0,001$) com efeito máximo ($\delta = 1,0$), indicando ausência prática de sobreposição entre as distribuições das amostras.

Esse comportamento discrepante é ilustrado graficamente na Figura 6, evidenciando que enquanto o paradigma *batch* cresce em escala de milhões de registros processados por segundo, o *streaming* é severamente limitado pelos gargalos inerentes da arquitetura de mensageria contínua. Os custos adicionais estão associados à sobrecarga (*overhead*) de rede e serialização no recebimento contínuo do Apache Kafka, coordenação e controle temporal de micro-lotes

Tabela 9 – Comparação de throughput e testes de significância não-paramétricos (RQ1).

Comparação	Batch Médio (M linhas/s)	Stream Médio (linhas/s)	Valor- <i>p</i>	Tamanho de Efeito (δ)
BL1 × SL1 (200MB)	1,76	159,18	< 0,001	Grande (1,0)
BL2 × SL2 (1GB)	6,87	374,26	< 0,001	Grande (1,0)
BL3 × SL3 (3GB)	13,44	741,22	< 0,001	Grande (1,0)
BL4 × SL4 (10GB)	20,74	1274,14	< 0,001	Grande (1,0)

Fonte: elaborado pelo autor.

mínimos (*micro-batching*) e mecanismos internos de tolerância a falhas por *checkpointing* incremental.

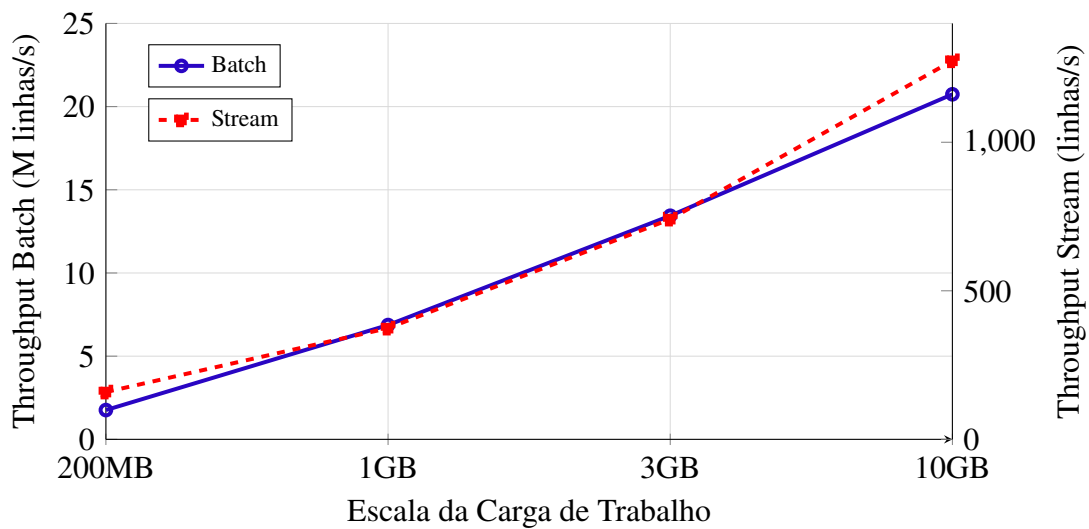


Figura 6 – Variações de throughput sob escalonamento de carga de trabalho (RQ1).

Os resultados apresentados na Figura 6 indicam comportamentos distintos entre os paradigmas analisados. Enquanto o processamento em *batch* apresenta uma curva de crescimento acentuada conforme a carga aumenta, o modelo de *stream* exibe estabilização precoce devido ao custo de coordenação das janelas de tempo. A análise estatística confirma que a diferença de desempenho observada é estatisticamente significativa ($p < 0,001$), validando a hipótese de que o dimensionamento da carga de trabalho afeta diretamente a eficiência das arquiteturas de processamento distribuído.

6.3.2 RQ2 — Impacto do Volume de Dados e Variabilidade Operacional

Na segunda questão de pesquisa experimental (RQ2), investigou-se em que medida o escalonamento do volume de dados bruto afeta o desempenho interno e a estabilidade de cada paradigma de processamento. Para validar a significância estatística do impacto volumétrico sem assumir premissas de normalidade nas distribuições amostrais, aplicou-se o teste global não-paramétrico de Kruskal-Wallis. O teste confirmou com alto grau de confiança que o volume

dos dados exerce um impacto estatisticamente significativo sobre a vazão operacional em ambos os cenários de arquitetura ($p < 0,001$).

Como o teste de Kruskal-Wallis aponta apenas a existência de diferença global, executaram-se análises *post-hoc* utilizando o teste de comparações múltiplas de Dunn, aplicando a correção rígida de Bonferroni para mitigar a ocorrência de falsos positivos (erros do Tipo I). Os resultados dos testes par a par ratificaram que todas as transições de escala de dados (e.g., a passagem de 200MB para 1GB, e sucessivamente até 10GB) produzem variações de desempenho isoladamente significativas em ambos os ambientes computacionais.

A Tabela 10 expõe os resultados estruturados a partir do Coeficiente de Variação ($CV\%$), revelando o comportamento da dispersão relativa dos dados sob estresse volumétrico.

Tabela 10 – Análise de variabilidade operacional sob escalonamento volumétrico (RQ2).

(a) Paradigma Batch			(b) Paradigma Stream		
Cenário	Média (M linhas/s)	CV (%)	Cenário	Média (linhas/s)	CV (%)
BL1 (200MB)	1,76	4,12	SL1 (200MB)	159,18	8,45
BL2 (1GB)	6,87	2,85	SL2 (1GB)	374,26	7,90
BL3 (3GB)	13,44	1,94	SL3 (3GB)	741,22	9,12
BL4 (10GB)	20,74	1,10	SL4 (10GB)	1274,14	11,34

Fonte: elaborado pelo autor.

A Figura 7 ilustra o comportamento do Coeficiente de Variação frente ao aumento da carga de trabalho, permitindo contrastar visualmente a dinâmica de estabilização de ambos os paradigmas.

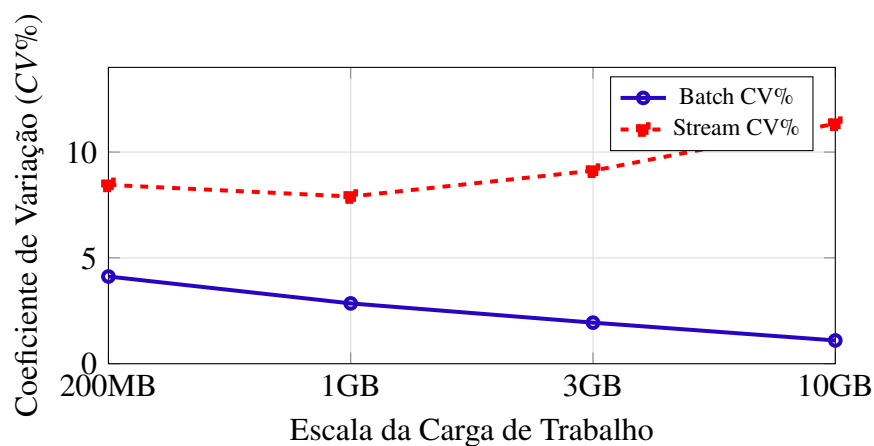


Figura 7 – Evolução da variabilidade operacional ($CV\%$) conforme a escala de carga (RQ2).

A análise conjunta dos dados e do comportamento gráfico revela um fenômeno de divergência estrutural entre as duas abordagens arquiteturais. No paradigma *batch*, constata-se que o aumento volumétrico atua como um fator de estabilização do cluster. À medida que a carga

de dados cresce de 200MB para 10GB, o valor do $CV\%$ decresce de forma contínua, caindo de 4,12% para meros 1,10%.

Este comportamento é explicado pela amortização eficiente dos custos fixos computacionais em sistemas distribuídos. Cargas muito pequenas sofrem uma penalização proporcional elevada decorrente do *overhead* de inicialização do ecossistema Spark, alocação de JVMs, registro de *executors* e coordenação inicial do *Task Scheduler*. Quando o volume de dados se torna massivo, o tempo gasto com o processamento real da carga domina a execução sobre os custos fixos de gerência, resultando em uma operação altamente previsível, estável e com baixa dispersão estatística.

Em contrapartida, o paradigma *streaming* exibe uma curva de instabilidade ascendente. Embora ocorra uma leve oscilação positiva na transição inicial para 1GB (7,90%), o avanço para volumes de 3GB e 10GB eleva o Coeficiente de Variação para 9,12% e 11,34%, respectivamente.

Este ganho de volatilidade indica que o escalonamento volumétrico em pipelines contínuos penaliza a consistência temporal da computação. Sob cargas elevadas, os nós do cluster enfrentam gargalos cumulativos na camada de rede (ingestão via tópicos do Apache Kafka), saturação de memória física durante as operações de agrupamento em micro-lotes e atrasos recorrentes na persistência dos metadados de *checkpointing*. Essa disputa intensa por recursos computacionais gera picos localizados de latência e variações de processamento entre as janelas temporais, o que degrada a estabilidade macroscópica do sistema e eleva a incerteza operacional da arquitetura de fluxo contínuo.

6.3.3 RQ3 — Intervalos de Trigger e Latência de Eventos

Na terceira questão de pesquisa experimental (RQ3), restringiu-se o escopo de análise ao ecossistema de processamento contínuo (*streaming*) com o objetivo de mensurar as consequências operacionais da parametrização temporal dos gatilhos de execução (*triggers*). Para determinar o impacto dessas variações de configuração, aplicou-se o teste não-paramétrico de Kruskal-Wallis. O modelo estatístico validou de forma robusta que a alteração programática do intervalo de *trigger* resulta em modificações estatisticamente significativas nas métricas de latência e estabilidade do *Spark Structured Streaming* ($p < 0,001$).

A Tabela 11 consolida as variáveis de desempenho capturadas durante as sessões controladas sob os intervalos amostrais de 1s, 2s e 5s.

A Figura 8 ilustra graficamente o comportamento oposto da latência do evento em relação ao tempo de barramento reprimido (*backpressure*), servindo como subsídio visual para o mapeamento de gargalos.

Os dados empíricos e as curvas expostas revelam um *trade-off* crítico de design arquitetural para engenharia de software de fluxos de dados. Observa-se que a dilatação deliberada do intervalo de gatilho de 1s para 5s atua de forma eficaz na mitigação da pressão imediata exercida

Tabela 11 – Métricas operacionais de streaming segmentadas por intervalo de trigger (RQ3).

Cenário	Lat. Trigger (ms)	Lat. Evento (s)	Backpressure (ms)	Throughput (linhas/s)	Jitter (ms)
ST1 (1s)	1464,70	1,42	240,10	745,20	75,90
ST2 (2s)	1828,00	2,15	115,40	742,10	122,90
ST3 (5s)	2571,00	5,60	42,80	739,80	152,60

Fonte: elaborado pelo autor.

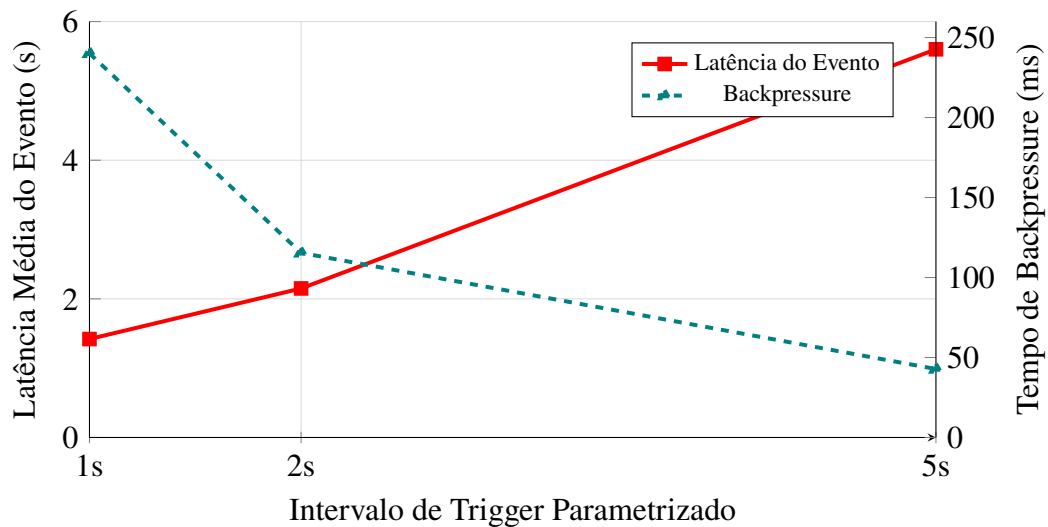


Figura 8 – Dilema arquitetural (trade-off) entre latência do evento e backpressure operacional (RQ3).

sobre as filas de ingestão do sistema. Esse fenômeno é evidenciado pela redução drástica no tempo médio de interrupção por *backpressure*, que cai de 240,10ms para meros 42,80ms. Ao espaçar as execuções, o motor de micro-lotes ganha janelas mais folgadas para processar as mensagens retidas no Apache Kafka, reduzindo a concorrência imediata de escrita e o estresse sobre os buffers internos de memória do cluster.

Contudo, este alívio operacional impõe uma penalidade severa sobre os requisitos não-funcionais de reatividade e previsibilidade temporal do sistema. A latência média acumulada do evento sofre uma degradação expressiva, saltando de 1,42s para 5,60s. Esse atraso é acompanhado por uma ampliação acentuada na instabilidade temporal da entrega dos dados (*jitter*), que duplica de 75,90ms para 152,60ms, sinalizando um comportamento de processamento altamente irregular e em rajadas (*bursty*).

Adicionalmente, constata-se que a estratégia de acumular um volume maior de registros por ciclo (*micro-batching* expandido) não se reverteu em ganhos de eficiência de vazão computacional, uma vez que o *throughput* bruto permaneceu praticamente estagnado na faixa de 740 linhas por segundo em todos os cenários. Do ponto de vista de design de arquitetura de software, essa descoberta demonstra que a parametrização do *trigger* não funciona como uma

alavanca para o aumento de capacidade de vazão máxima, configurando-se estritamente como um botão de sintonia fina para equilibrar a tolerância a picos de carga contra a necessidade de processamento em tempo real estrito.

6.3.4 RQ4 — Escalabilidade Horizontal e Eficiência Arquitetural

Na quarta questão de pesquisa (RQ4), avaliou-se a capacidade de resposta de ambos os paradigmas diante da expansão da infraestrutura computacional utilizando clusters com 1, 2 e 4 instâncias operacionais (*workers*) sob a carga máxima de 10GB. Para quantificar a eficiência de paralelização, foram computadas as métricas de Speedup Observado (S) e Eficiência de Escalonamento (E), cujos resultados estão expostos na Tabela 12.

Tabela 12 – Métricas de escalabilidade e eficiência de paralelização no cluster (RQ4).

Paradigma	Workers	Throughput Médio	Comportamento	Speedup (S)	Eficiência (E)
Batch	1	20,67 M linhas/s	Linha de Base	1,00	1,00
	2	29,67 M linhas/s	Sublinear	1,44	0,72
	4	30,92 M linhas/s	Saturação	1,50	0,38
Stream	1	1264,00 linhas/s	Linha de Base	1,00	1,00
	2	1220,00 linhas/s	Degradação	0,96	0,48
	4	1172,00 linhas/s	Ineficiência	0,93	0,23

Fonte: elaborado pelo autor.

A divergência estrutural entre as duas abordagens é evidente. O processamento *batch* demonstra ganho positivo de escalabilidade horizontal ao expandir de 1 para 2 *workers* ($S = 1,44$). Contudo, atinge um patamar de saturação precoce em 4 *workers* ($S = 1,50$), onde a eficácia computacional cai para 38%, indicando que a sobrecarga distribuída de particionamento e coordenação passa a anular os benefícios do acréscimo de hardware.

No paradigma de *streaming*, constata-se um cenário de escalabilidade negativa. A inserção de novos nós de processamento causou regressão no *throughput* absoluto (reduzindo de 1264 para 1172 linhas/s em 4 *workers*), derrubando a eficiência do cluster para pífios 23%. Esse comportamento deletério indica que a sobrecarga contínua associada à coordenação dos estágios, sincronização de *offsets* no Apache Kafka e gestão de estado distribuído sob o modelo de micro-lotes impõe barreiras severas ao paralelismo horizontal. Esse fenômeno é mapeado comparativamente contra a linha ideal linear na Figura 9.

A Tabela 13 sintetiza as descobertas obtidas até o momento nas frentes de investigação mapeadas, servindo como base estrutural para a consolidação do modelo conceitual.

O mapeamento empírico consolidado na Tabela 13 evidencia que a engenharia de sistemas intensivos em dados não pode se apoiar em premissas simplistas de escalonamento linear. Os comportamentos observados nas quatro questões de pesquisa demonstram que variáveis de

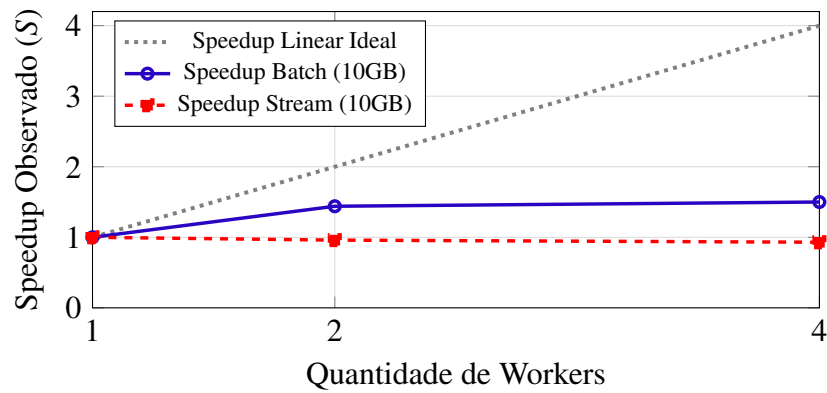


Figura 9 – Curvas de speedup experimental frente ao limite linear ideal (RQ4).

Tabela 13 – Síntese dos resultados parciais e implicações arquiteturais.

Dimensão Analisada	Resultado Parcial Consolidado	Implicação para o Modelo de Decisão
Revisão Sistemática	10 estudos primários preencheram integralmente os critérios de elegibilidade e qualidade.	O volume reduzido evidencia a fragmentação de dados concretos para o design de sistemas intensivos em dados.
Síntese Quantitativa	As otimizações arquiteturais exibiram efeito médio positivo ($\overline{\text{LRR}} = 0,981$), com razão de resposta de $\approx 2,7\times$.	Há indícios robustos de ganhos empíricos, mas as evidências na literatura carecem de padronização nos relatos estatísticos.
RQ1 — Batch vs Streaming	O paradigma <i>batch</i> demonstrou superioridade de vazão em ordens de magnitude na comparação direta.	A escolha do paradigma deve priorizar os requisitos não-funcionais: maximização de throughput bruto (<i>batch</i>) vs menor latência temporal (<i>stream</i>).
RQ2 — Volume de Dados	O volume determinou a estabilização estatística do <i>batch</i> ($CV = 1,10\%$) e o aumento de instabilidade do <i>stream</i> ($CV = 11,34\%$).	O volume de dados bruto deve atuar como variável de entrada ponderada na matriz de decisão da arquitetura.
RQ3 — Triggers	O escalonamento do trigger (1s para 5s) mitigou o backpressure (240ms para 42ms), mas dobrou o jitter (75ms para 152ms).	Configurações de micro-lotes introduzem trade-offs matemáticos inevitáveis entre estabilidade temporal e vazão.
RQ4 — Escalabilidade	A adição de workers saturou o <i>batch</i> ($E = 0,38$) e causou regressão de desempenho no <i>streaming</i> ($E = 0,23$).	A infraestrutura horizontal não garante ganhos automáticos e depende da sensibilidade do paradigma ao overhead distribuído.

Fonte: elaborado pelo autor.

infraestrutura (como a adição de *workers*) e de configuração de *runtime* (como intervalos de *trigger*) desencadeiam reações severas e frequentemente contra-intuitivas na estabilidade e na vazão das arquiteturas distribuídas. A constatação de uma eficiência de paralelização decrescente no modelo *batch* ($E = 0,38$) e abertamente regressiva no modelo de *streaming* ($E = 0,23$) corrobora a necessidade de uma abordagem analítica formal para guiar decisões de design de software antes da alocação massiva de recursos computacionais.

Essa interdependência de fatores técnicos justifica a relevância de se unificar os achados empíricos em um arcabouço metodológico estruturado. Os dados de variabilidade da RQ2 e os *trade-offs* temporais da RQ3 isolam os limiares operacionais onde cada paradigma deixa de ser eficiente. Consequentemente, essas métricas deixam de ser meros indicadores de desempenho passivos e passam a atuar como os parâmetros de entrada quantitativos indispensáveis para a calibração do modelo de decisão arquitetural proposto nesta pesquisa.

6.4 Integração dos Resultados

Os resultados parciais da revisão sistemática e da campanha experimental convergem para uma mesma conclusão: decisões arquiteturais em sistemas intensivos em dados dependem fortemente do contexto. A literatura mostra que diferentes técnicas podem produzir ganhos relevantes, mas tais ganhos estão intrinsecamente associados a características específicas de *workloads*, ambientes e métricas escolhidas. A campanha experimental, por sua vez, comprova que a escolha entre *batch* e *streaming*, bem como a configuração de paralelismo e *trigger*, produz efeitos distintos sobre *throughput*, latência, escalabilidade e estabilidade.

Essa convergência sustenta a necessidade de uma abordagem de apoio à decisão arquitetural. Em vez de assumir que uma técnica ou paradigma é universalmente superior, a decisão deve ponderar as características dos dados, os requisitos de desempenho e as restrições operacionais do sistema.

A principal contribuição dos resultados parciais reside na identificação de relações entre fatores arquiteturais e métricas observáveis. A Revisão Sistemática da Literatura indica que ganhos de desempenho podem ser obtidos em diferentes camadas do sistema. A campanha experimental demonstra que decisões como escolha do paradigma, variação do volume, ajuste de *trigger* e aumento do número de *workers* geram efeitos discrepantes dependendo do contexto operacional.

6.5 Implicações para o Modelo de Apoio à Decisão

Os resultados obtidos até o momento fornecem insumos fundamentais para a especificação do modelo proposto. A Revisão Sistemática da Literatura indica quais tipos de decisões arquiteturais têm sido formalmente investigadas e quais métricas são adotadas para avaliá-las. A

campanha experimental fornece evidências controladas sobre como fatores como volume, taxa de eventos, paralelismo e intervalo de *trigger* degradam ou otimizam o comportamento de um *pipeline* baseado em Apache Spark e Apache Kafka.

Esses resultados sugerem que o modelo de apoio à decisão deve incorporar múltiplas dimensões, destacando-se:

- **Características dos dados:** tais como volume, distribuição, redundância e entropia;
- **Características da carga:** como taxa de eventos e variabilidade temporal;
- **Requisitos arquiteturais:** incluindo latência máxima aceitável e *throughput* mínimo esperado;
- **Configurações operacionais:** como número de *workers* e intervalo de *trigger*;
- **Métricas de custo computacional:** tais como perfil de uso de CPU, memória, rede e operações de disco.

Dessa forma, os resultados parciais não apenas respondem às questões iniciais da revisão e da campanha experimental, mas auxiliam na delimitação dos elementos que deverão compor a próxima etapa metodológica da pesquisa. Esses achados reforçam a necessidade de uma abordagem de apoio à decisão arquitetural que considere evidências empíricas, características intrínsecas dos dados e requisitos de desempenho, avançando além de recomendações genéricas ou dependentes exclusivamente da intuição do arquiteto de software.

7

Cronograma

O cronograma previsto para a continuidade da pesquisa estende-se até a defesa da dissertação, prevista para julho de 2027. O planejamento considera que parte das etapas iniciais da pesquisa já foi desenvolvida, incluindo a Revisão Sistemática da Literatura, a submissão de artigo derivado dessa etapa e a condução preliminar da campanha experimental.

As atividades restantes concentram-se no refinamento dos resultados experimentais, no aprofundamento da análise estatística, no cálculo de características informacionais dos dados, na construção do modelo quantitativo de apoio à decisão arquitetural, na validação preliminar da abordagem e na redação final da dissertação.

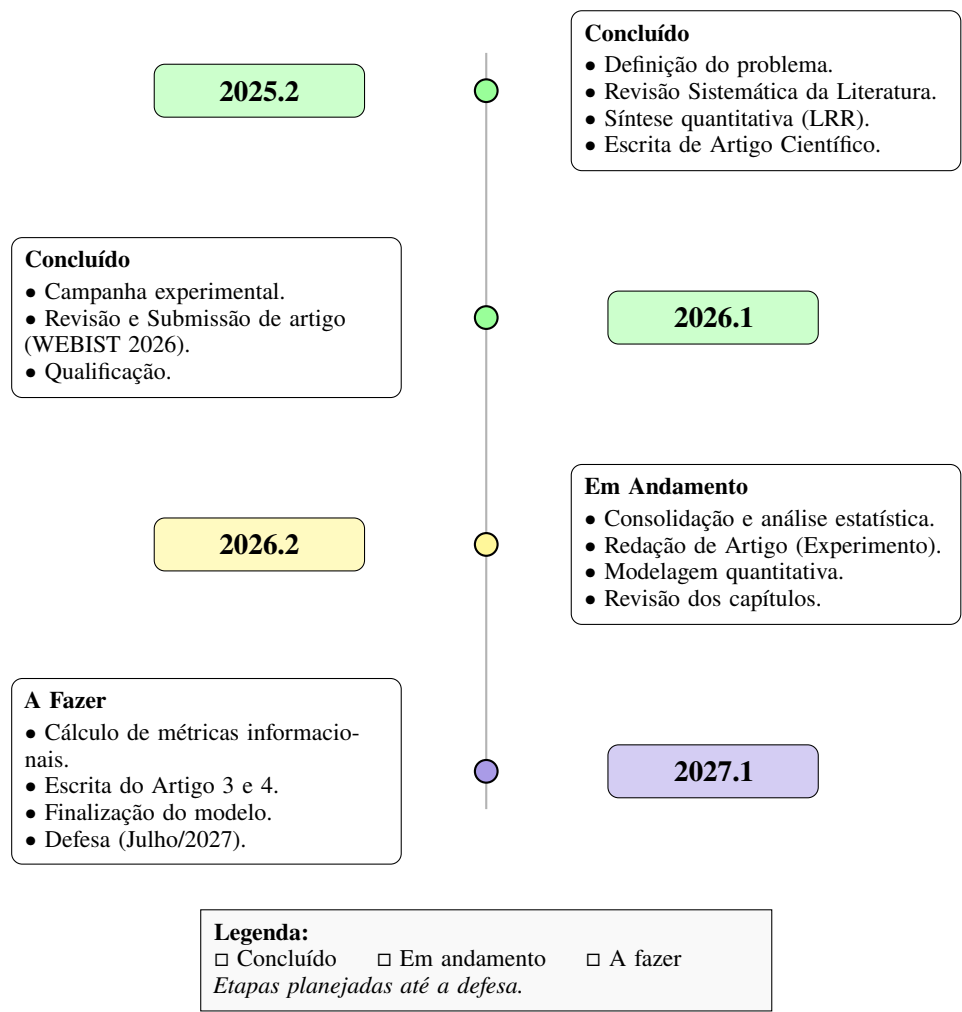


Figura 10 – Cronograma das atividades de pesquisa.

8

Próximos Passos

Esta pesquisa já possui três componentes principais desenvolvidos: a Revisão Sistemática da Literatura (RSL) e a campanha experimental comparando processamento *batch* e *streaming*; ou em desenvolvimento: a verificação de condução de mais experimentos na campanha inicial e a formulação inicial de uma abordagem quantitativa de apoio à decisão arquitetural. Os próximos passos concentram-se no refinamento dessa abordagem, no cálculo de características informacionais dos dados, na integração destas aos resultados experimentais e na validação preliminar do modelo.

8.1 Atividades Necessárias para a Conclusão da Pesquisa

Os próximos passos da pesquisa estão relacionados à transformação dos resultados já obtidos em uma contribuição integrada. A intenção é que a dissertação apresente uma abordagem articulada de apoio à tomada de decisão arquitetural em sistemas intensivos em dados. As atividades previstas são apresentadas na Tabela 14.

8.2 Refinamento do Modelo de Apoio à Decisão Arquitetural

O modelo deverá considerar características associadas aos dados e ao processamento, como volume, taxa de eventos, entropia, redundância, cardinalidade, variabilidade, *throughput*, latência e *backpressure*. Um cenário de processamento é representado pelo vetor de características:

$$C_D = \{V, \lambda, H, R, K, \sigma, L_{req}\} \quad (8.1)$$

em que V é o volume, λ a taxa de eventos, H a entropia, R a redundância, K a cardinalidade, σ a variabilidade e L_{req} a restrição de latência. Para cada alternativa arquitetural a , associa-se o conjunto de métricas $P_a = \{T_a, L_a, U_{cpu,a}, U_{mem,a}, B_a, E_a\}$. De forma exploratória, o escore de

Tabela 14 – Atividades previstas para a continuidade da pesquisa.

Atividade	Descrição	Resultado esperado
Consolidação experimental	Organizar tabelas, gráficos e resultados estatísticos.	Base de dados consolidada.
Cálculo de métricas informacionais	Calcular entropia, redundância, cardinalidade e variabilidade.	Caracterização informacional.
Refinamento do modelo	Definir entradas, saídas e critérios de ranqueamento.	Modelo quantitativo inicial.
Integração de resultados	Relacionar dados com métricas de desempenho.	Evidências de suporte.
Validação preliminar	Comparar recomendações com resultados observados.	Avaliação de coerência.
Redação final	Consolidar capítulos, revisar escrita e normas.	Versão final para defesa.

Fonte: elaborado pelo autor.

uma alternativa pode ser representado por:

$$S(a) = w_1 T'_a + w_2 L'_a + w_3 U'_a + w_4 E'_a \quad (8.2)$$

Essa formulação será refinada conforme a análise dos dados experimentais e a definição das métricas mais relevantes.

8.3 Cálculo das Características Informacionais dos Dados

Utilizando a Teoria da Informação, serão calculadas métricas como a entropia de Shannon:

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (8.3)$$

e a redundância:

$$R(X) = 1 - \frac{H(X)}{H_{max}(X)} \quad (8.4)$$

Essas métricas serão analisadas em conjunto com os dados experimentais, buscando correlações entre a estrutura da informação e o comportamento das arquiteturas sob carga.

8.4 Produção Científica Prevista

Além da dissertação, a pesquisa prevê a produção de artigos conforme a Tabela 15.

8.5 Delimitação do Escopo e Trabalhos Futuros

O escopo está delimitado ao uso da RSL, da campanha experimental já conduzida e da validação preliminar do modelo. Trabalhos futuros poderão ampliar a pesquisa através da:

Tabela 15 – Produção científica prevista.

Artigo	Tema	Situação	Período
Artigo 1	RSL e Design de Sistemas	Submetido (WEBIST)	2026.1
Artigo 2	Campanha Experimental	Em elaboração	2026.2
Artigo 3	Modelo de Decisão	Planejado	2026.2–2027.1
Artigo 4	Consolidação e Validação	Planejado	2027.1

Fonte: elaborado pelo autor.

- Avaliação com outros *datasets* e tecnologias (e.g., Flink, Kafka Streams);
- Inclusão de métricas de custo financeiro, consumo energético e resiliência;
- Validação em ambientes de produção real;
- Desenvolvimento de ferramenta de automação para recomendações arquiteturais.

Referências Bibliográficas

AHMED, N. et al. A comprehensive performance analysis of apache hadoop and apache spark for large scale data sets using hibench. *Journal of Big Data*, v. 7, n. 110, 2020. Citado 2 vezes nas páginas 19 e 21.

ALCANTARA, J. *Performance Evaluation of a Big Data Application on Apache Spark*. Dissertação (Master of Engineering Project) — Ryerson University, Toronto, Ontario, Canada, 2019. Citado na página 19.

ALQUWAIEE, H.; WU, C. On performance modeling and prediction for spark-hbase applications in big data systems. In: *ICC 2022 – IEEE International Conference on Communications*. [S.l.: s.n.], 2022. p. 3685–3690. Citado 2 vezes nas páginas 34 e 46.

ARMBRUST, M. et al. Structured streaming: A declarative api for real-time applications in apache spark. In: *Proceedings of the 2018 International Conference on Management of Data*. New York, NY, USA: Association for Computing Machinery, 2018. (SIGMOD '18), p. 601–613. Citado 5 vezes nas páginas 13, 18, 19, 24 e 38.

AUNG, T.; MIN, H. Y.; MAW, A. H. Coordinate checkpoint mechanism on real-time messaging system in kafka pipeline architecture. In: *Proceedings of AITC 2019*. [S.l.: s.n.], 2019. p. 37–42. Booktitle a conferir. Citado 5 vezes nas páginas 12, 13, 20, 34 e 46.

AWAN, A. J. et al. How data volume affects spark based data analytics on a scale-up server. In: *Big Data Benchmarking*. [S.l.]: Springer, 2015, (Lecture Notes in Computer Science, v. 9495). p. 81–92. Citado 2 vezes nas páginas 19 e 21.

CLIFF, N. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, v. 114, n. 3, p. 494–509, 1993. Citado 2 vezes nas páginas 26 e 44.

DAI, H.-N. et al. Big data analytics for large scale wireless networks: Challenges and opportunities. *EBSE Technical Report*, ACM New York, NY, USA, v. 52, n. 5, p. 1–36, 2019. Citado 2 vezes nas páginas 12 e 16.

DUNN, O. J. Multiple comparisons using rank sums. *Technometrics*, v. 6, n. 3, p. 241–252, 1964. Citado 2 vezes nas páginas 26 e 43.

GÖTZE, P.; BAUMANN, S.; SATTLER, K.-U. An nvm-aware storage layout for analytical workloads. In: *2018 IEEE 34th International Conference on Data Engineering Workshops (ICDEW)*. [S.l.: s.n.], 2018. p. 110–115. Citado 3 vezes nas páginas 17, 34 e 46.

HAYNES, B.; CHEUNG, A.; BALAZINSKA, M. Pipegen: Data pipe generator for hybrid analytics. In: *Proceedings of the Seventh ACM Symposium on Cloud Computing*. New York, NY, USA: Association for Computing Machinery, 2016. (SoCC '16), p. 470–483. ISBN 9781450345255. Citado 5 vezes nas páginas 12, 13, 17, 34 e 46.

HILBERT, M. Big data for development: A review of promises and challenges. *Development Policy Review*, Wiley Online Library, v. 34, n. 1, p. 135–174, 2016. Citado 2 vezes nas páginas 12 e 16.

IVANOV, T.; TAAFFE, J. Exploratory analysis of spark structured streaming. In: *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. New York, NY, USA: Association for Computing Machinery, 2018. (ICPE '18), p. 141–146. Citado 2 vezes nas páginas 18 e 19.

JI, S. et al. Quality assurance technologies of big data applications: A systematic literature review. *Applied Sciences*, MDPI, v. 10, n. 22, p. 8052, 2020. Citado 2 vezes nas páginas 12 e 17.

KITCHENHAM, B. et al. Systematic literature reviews in software engineering – a systematic literature review. *Information and Software Technology*, Elsevier, v. 51, n. 1, p. 7–15, 2009. Citado 2 vezes nas páginas 24 e 28.

KITCHENHAM, B.; CHARTERS, S. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. [S.l.], 2007. Citado 2 vezes nas páginas 24 e 28.

KITCHENHAM, B. A.; BUDGEN, D.; BRERETON, P. *Evidence-based software engineering and systematic reviews*. [S.l.]: CRC Press, 2015. v. 4. Citado 3 vezes nas páginas 23, 24 e 28.

KLEPPMANN, M. *Designing Data-Intensive Applications*. [S.l.]: O'Reilly Media, 2017. Citado 2 vezes nas páginas 12 e 16.

KREPS, J.; NARKHEDE, N.; RAO, J. Kafka: A distributed messaging system for log processing. In: *Proceedings of the NetDB Workshop*. [S.l.: s.n.], 2011. Citado 4 vezes nas páginas 13, 20, 24 e 39.

KRUSKAL, W. H.; WALLIS, W. A. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*, v. 47, n. 260, p. 583–621, 1952. Citado 2 vezes nas páginas 26 e 43.

LI, Z. et al. Karst: Transactional data ingestion without blocking on a scalable architecture. *IEEE Transactions on Knowledge and Data Engineering*, v. 34, n. 5, p. 2241–2253, 2022. Citado 3 vezes nas páginas 17, 34 e 46.

LYU, C. et al. Fine-grained modeling and optimization for intelligent resource management in big data processing. *Proceedings of the VLDB Endowment*, VLDB Endowment, v. 15, n. 11, p. 3098–3111, 2022. ISSN 2150-8097. Citado 5 vezes nas páginas 12, 13, 17, 34 e 46.

MANN, H. B.; WHITNEY, D. R. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, v. 18, n. 1, p. 50–60, 1947. Citado 2 vezes nas páginas 26 e 43.

MATTEUSSI, K. J. et al. Performance evaluation analysis of spark streaming backpressure for data-intensive pipelines. *Sensors*, v. 22, n. 13, p. 4756, 2022. Citado 3 vezes nas páginas 13, 18 e 21.

MATTMANN, C. A. et al. Software architecture for large-scale, distributed, data-intensive systems. In: IEEE. *Proceedings of the Fourth Working IEEE/IFIP Conference on Software Architecture (WICSA 2004)*. [S.l.], 2004. p. 255–264. Citado 2 vezes nas páginas 12 e 17.

NASIRI, H.; NASEHI, S.; GOUDARZI, M. Evaluation of distributed stream processing frameworks for iot applications in smart cities. *Journal of Big Data*, v. 6, n. 52, 2019. Citado na página 18.

- PAGE, M. J. et al. The prisma 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, v. 372, p. n71, 2021. Citado 2 vezes nas páginas 24 e 28.
- PONCE, L. M. et al. Upgrading a high performance computing environment for massive data processing. *Journal of Internet Services and Applications*, v. 10, n. 19, 2019. Citado 2 vezes nas páginas 34 e 46.
- SCHIAVIO, F.; BONETTA, D.; BINDER, W. Dynamic speculative optimizations for sql compilation in apache spark. *Proceedings of the VLDB Endowment*, VLDB Endowment, v. 13, n. 5, p. 754–767, 2020. ISSN 2150-8097. Citado 3 vezes nas páginas 17, 34 e 46.
- SHANAHAN, J. G.; DAI, L. Large scale distributed data science using apache spark. In: *Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: Association for Computing Machinery, 2015. (KDD '15), p. 2323–2324. Citado na página 19.
- SHANNON, C. E. A mathematical theory of communication. *The Bell System Technical Journal*, v. 27, p. 379–423, 623–656, 1948. Reprinted with corrections. Citado 2 vezes nas páginas 21 e 25.
- SHAPIRO, S. S.; WILK, M. B. An analysis of variance test for normality: Complete samples. *Biometrika*, v. 52, n. 3/4, p. 591–611, 1965. Citado 2 vezes nas páginas 25 e 43.
- Shyamasundar L. B.; V. Anilkumar; Jhansi Rani P. Fine-tuning resource allocation of apache spark distributed multinode cluster for faster processing of network-trace data. *International Journal of Advanced Computer Science and Applications*, v. 10, n. 11, 2019. Citado na página 19.
- ULLAH, F. et al. Evaluation of distributed data processing frameworks in hybrid clouds. *Journal of Network and Computer Applications*, v. 224, p. 103837, 2024. Preprint originally available as arXiv:2201.01948. Citado na página 20.
- VENKATARAMAN, S. et al. Drizzle: Fast and adaptable stream processing at scale. In: *Proceedings of the 26th Symposium on Operating Systems Principles*. New York, NY, USA: ACM, 2017. (SOSP '17), p. 374–389. ISBN 978-1-4503-5085-3. Citado na página 18.
- VINÇON, T. et al. nkV in action: Accelerating kv-stores on native computational storage with near-data processing. *Proceedings of the VLDB Endowment*, VLDB Endowment, v. 13, n. 12, p. 2981–2984, 2020. ISSN 2150-8097. Citado 3 vezes nas páginas 17, 34 e 46.
- WANG, G. et al. A hard real-time scheduler for spark on yarn. In: *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. [S.l.]: IEEE Press, 2018. (CCGrid '18), p. 645–652. ISBN 9781538658154. Citado 5 vezes nas páginas 12, 13, 17, 34 e 46.
- WOHLIN, C. et al. *Experimentation in software engineering*. [S.l.]: Springer Science & Business Media, 2012. Citado 3 vezes nas páginas 24, 26 e 37.
- ZAHARIA, M. *An Architecture for Fast and General Data Processing on Large Clusters*. Tese (Doutorado) — University of California, Berkeley, 2016. Citado 4 vezes nas páginas 13, 19, 24 e 38.